LAPORAN PRAKTIKUM BASIS DATA LANJUT

MODUL 4: IMPLEMENTASI TRIGGER, AUDIT TRAIL, DAN INTEGRITAS DATA

Mata Kuliah: Pemrograman Basis Data

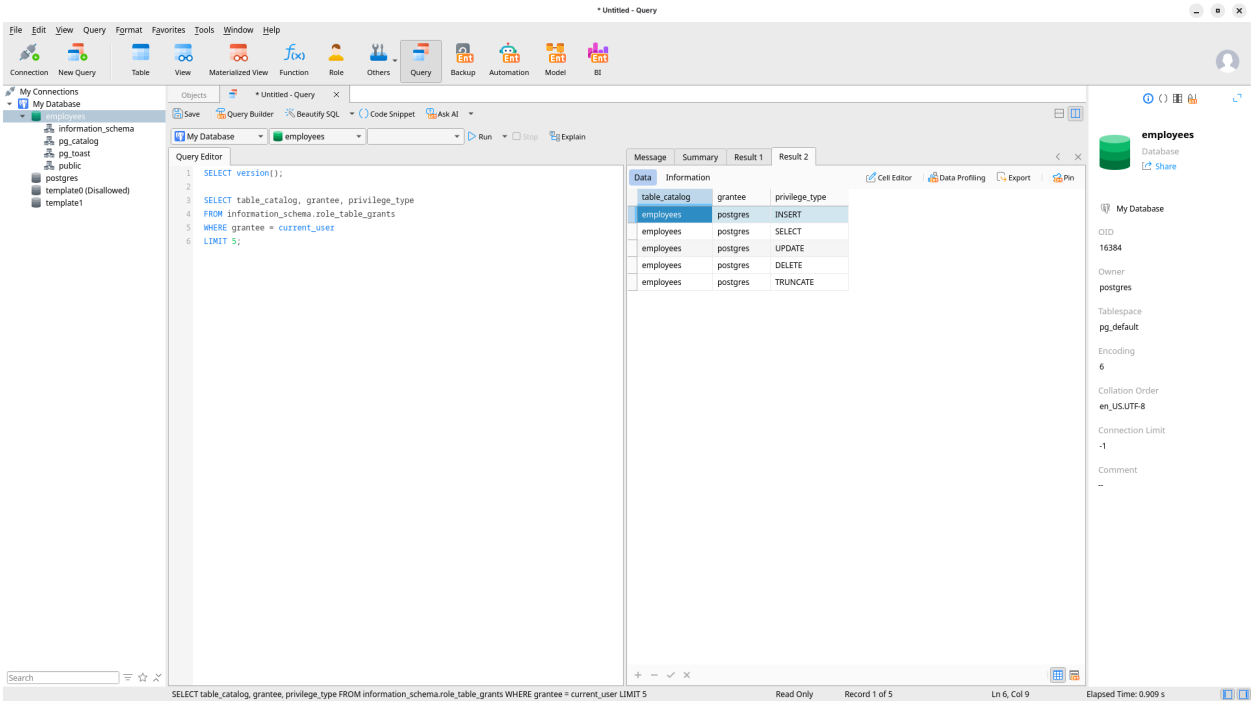
Nama : Muhromin

NIM : 25051214046

Bagian A - Persiapan dan Basis Data Kerja

2.1 Pemeriksaan Lingkungan

Pemeriksaan versi basis data dan hak akses pengguna (privilese) untuk memastikan dukungan pembuatan trigger dan fungsi PL/pgSQL.

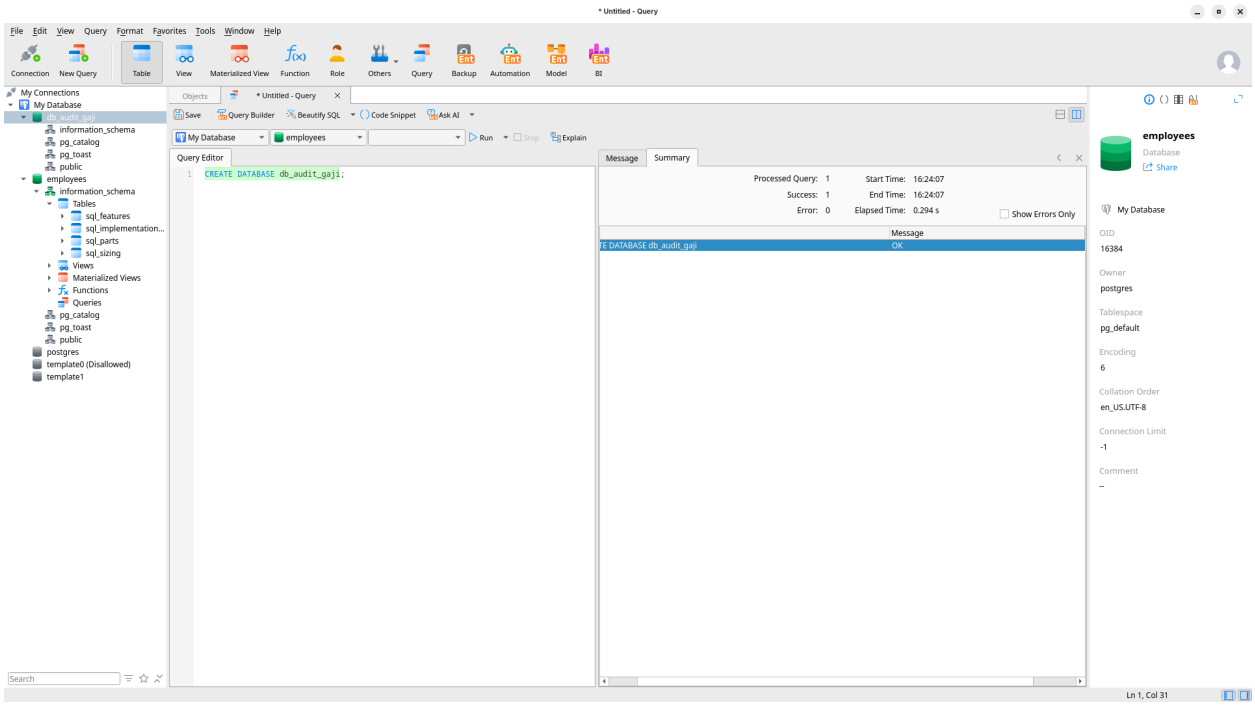


Pemeriksaan menunjukkan basis data berjalan pada PostgreSQL versi 18.6 x86_64 dengan akun postgres (superuser) yang memiliki hak akses penuh (INSERT, SELECT, UPDATE, DELETE, TRUNCATE, TRIGGER) pada katalog tabel basis data.

2.2 Membangun Basis Data Kerja (P1)

Pembuatan basis data baru khusus pengujian bernama db_audit_gaji agar tidak mengubah atau

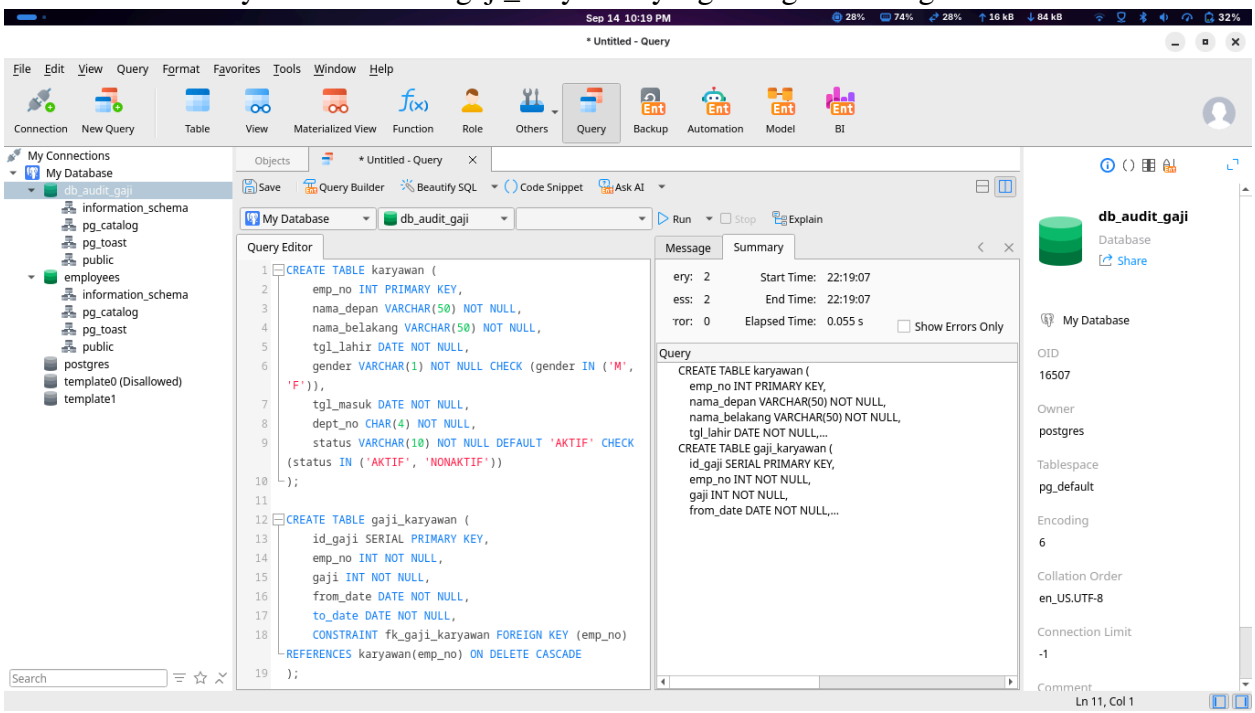
merusak basis data asli employees.



2.3 Rancang Tabel Operasional dan Pendukung (P2 & P4)

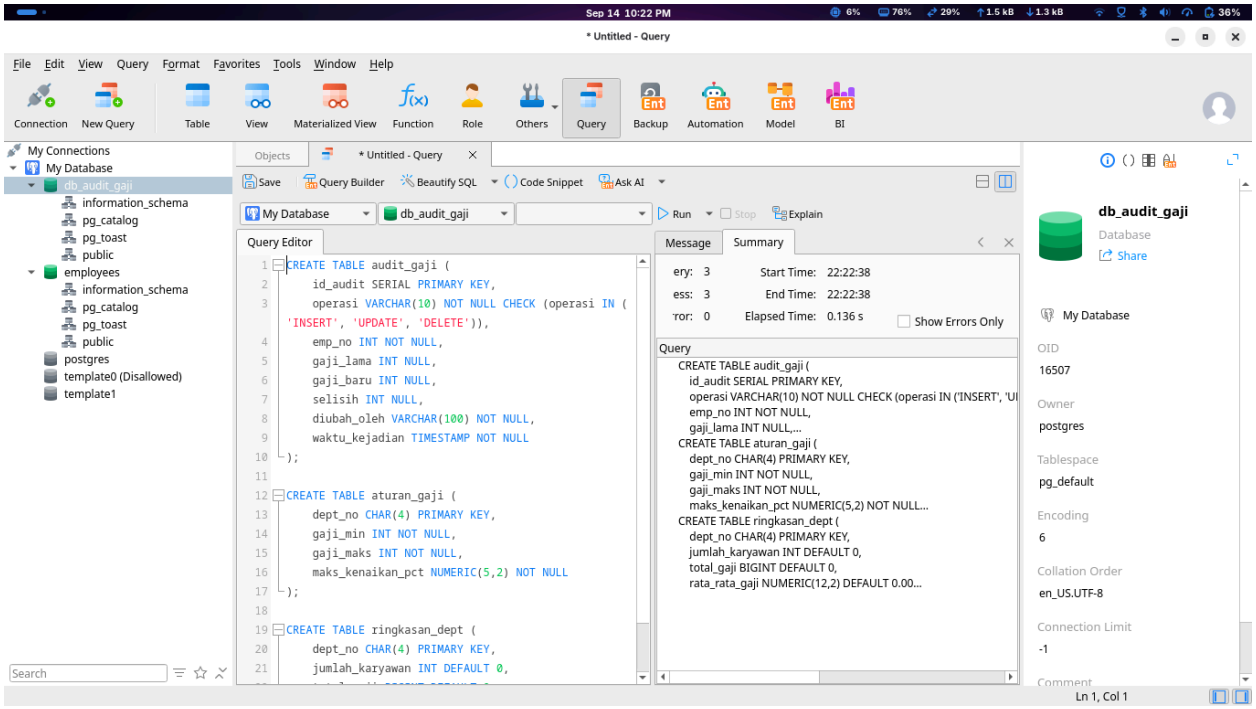
Tahap A.3 - Membuat 2 Tabel Operasional (P2)

Membuat tabel karyawan dan tabel gaji_karyawan yang saling terhubung.



Tahap A.4 - Membuat 3 Tabel Pendukung (P4)

Menyiapkan tabel riwayat audit, tabel aturan batas gaji, dan tabel ringkasan departemen.



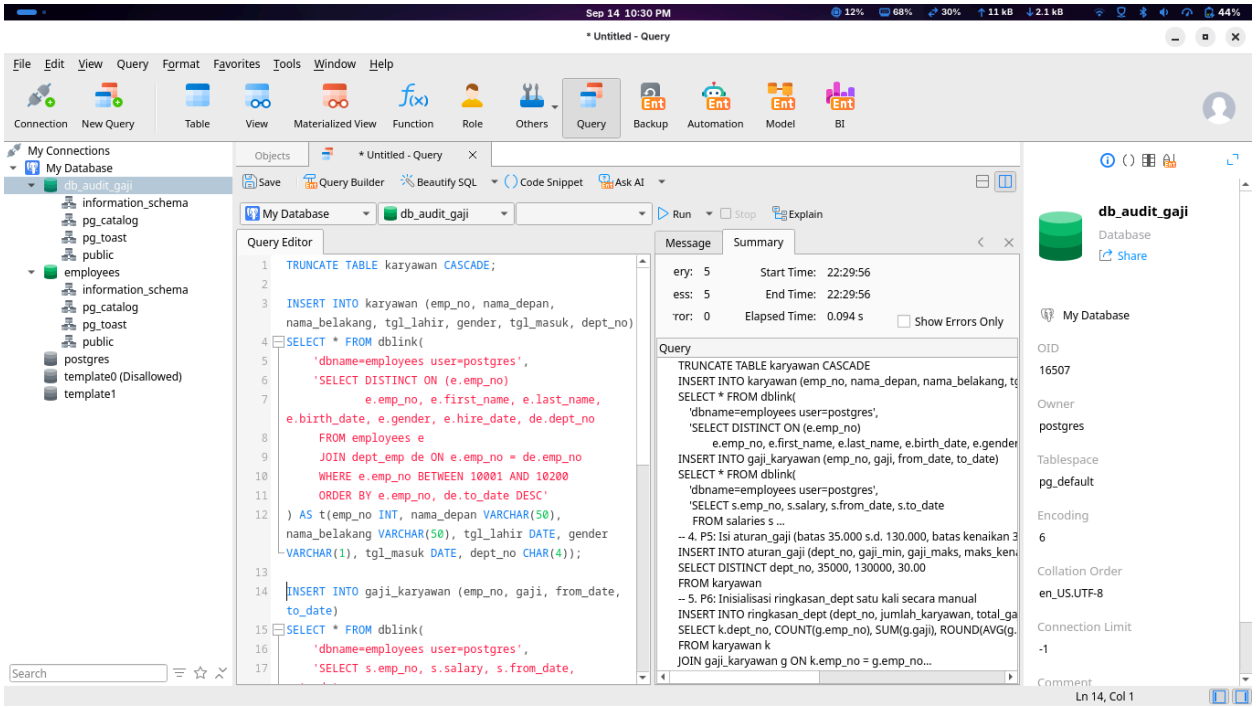

```
CREATE TABLE audit_gaji (
    id_audit SERIAL PRIMARY KEY,
    operasi VARCHAR(10) NOT NULL CHECK (operasi IN ('INSERT', 'UPDATE', 'DELETE')),
    emp_no INT NOT NULL,
    gaji_lama INT NULL,
    gaji_baru INT NULL,
    selisih INT NULL,
    diubah_oleh VARCHAR(100) NOT NULL,
    waktu_kejadian TIMESTAMP NOT NULL
);

CREATE TABLE aturan_gaji (
    dept_no CHAR(4) PRIMARY KEY,
    gaji_min INT NOT NULL,
    gaji_maks INT NOT NULL,
    maks_kenaikan_pct NUMERIC(5,2) NOT NULL
);

CREATE TABLE ringkasan_dept (
    dept_no CHAR(4) PRIMARY KEY,
    jumlah_karyawan INT DEFAULT 0,
```

Tahap A.5 - Migrasi Data Awal (P3, P5, P6)

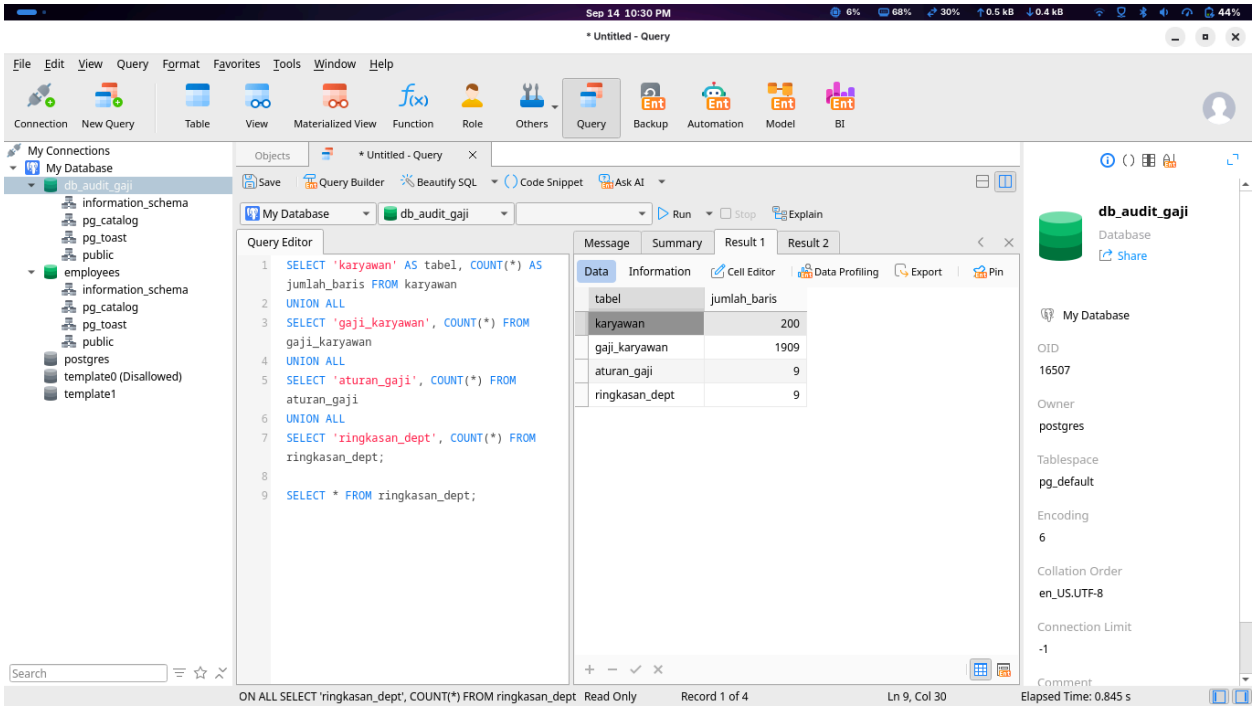
Menyalin sampel data karyawan dari database sebelah dan mengisi ringkasan awal.




```
TRUNCATE TABLE karyawan CASCADE;

INSERT INTO karyawan (emp_no, nama_depan,
nama_belakang, tgl_lahir, gender, tgl_masuk, dept_no)
SELECT * FROM dblink(
    'dbname=employees user=postgres',
    'SELECT DISTINCT ON (e.emp_no)
            e.emp_no, e.first_name, e.last_name,
e.birth_date, e.gender, e.hire_date, de.dept_no
     FROM employees e
     JOIN dept_emp de ON e.emp_no = de.emp_no
     WHERE e.emp_no BETWEEN 10001 AND 10200
     ORDER BY e.emp_no, de.to_date DESC'
) AS t(emp_no INT, nama_depan VARCHAR(50),
nama_belakang VARCHAR(50), tgl_lahir DATE, gender
VARCHAR(1), tgl_masuk DATE, dept_no CHAR(4));

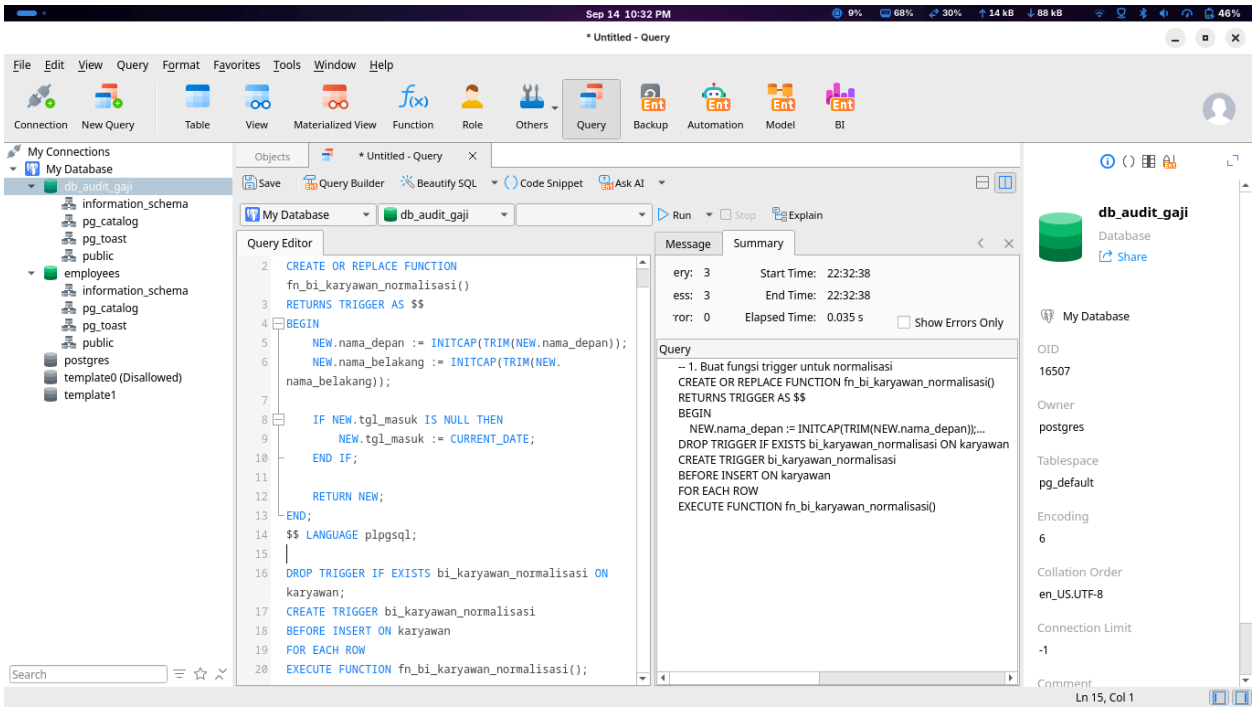
INSERT INTO gaji_karyawan (emp_no, gaji, from_date,
to_date)
SELECT * FROM dblink(
    'dbname=employees user=postgres',
    'SELECT s.emp_no, s.salary, s.from_date,
```



Bagian B - Trigger Validasi dan Normalisasi Data

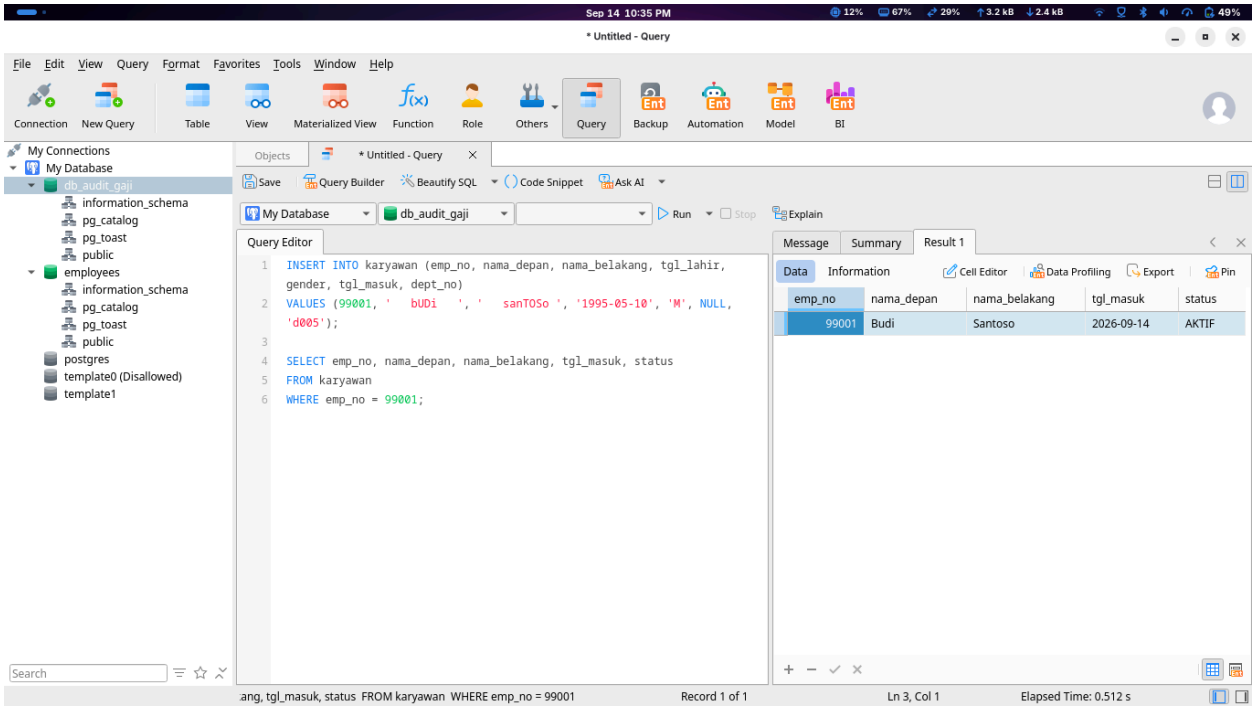
Tahap B.1 - Membuat Trigger Normalisasi

Membuat trigger agar huruf kapital nama depan dan belakang otomatis rapi saat disimpan.



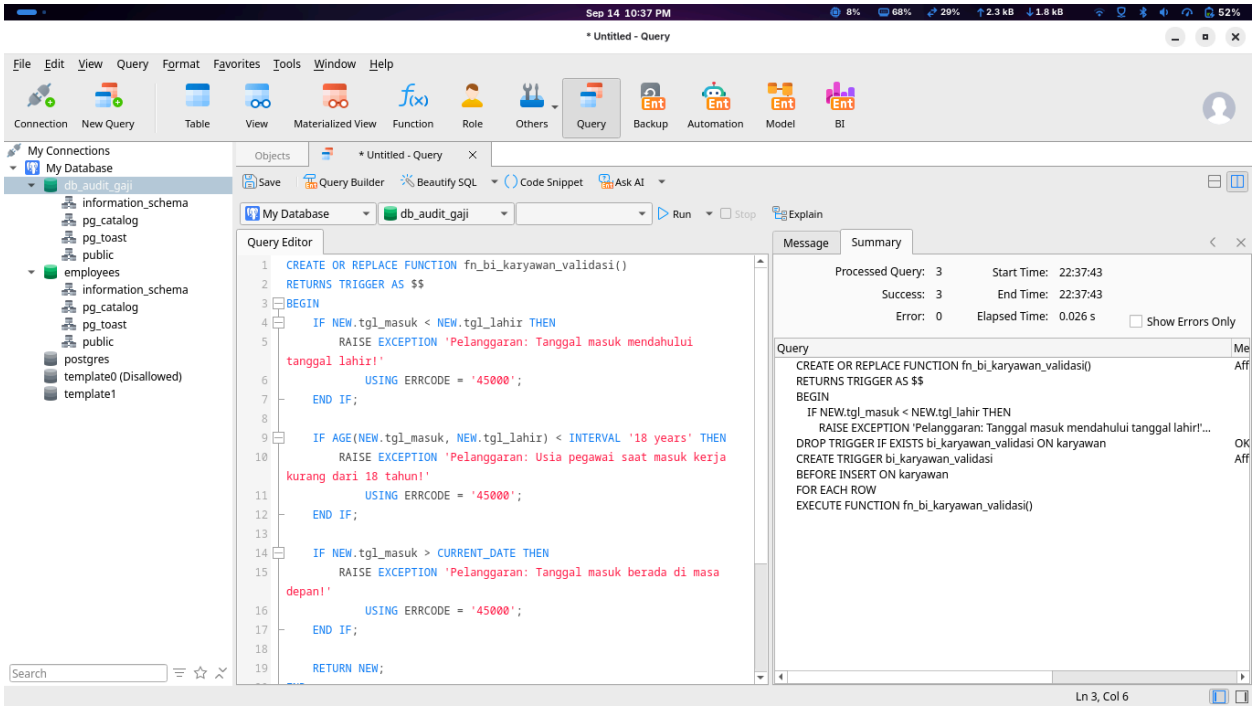
Tahap B.2 - Pengujian Trigger Normalisasi

Menguji input nama yang tulisannya berantakan untuk membuktikan hasilnya jadi rapi.



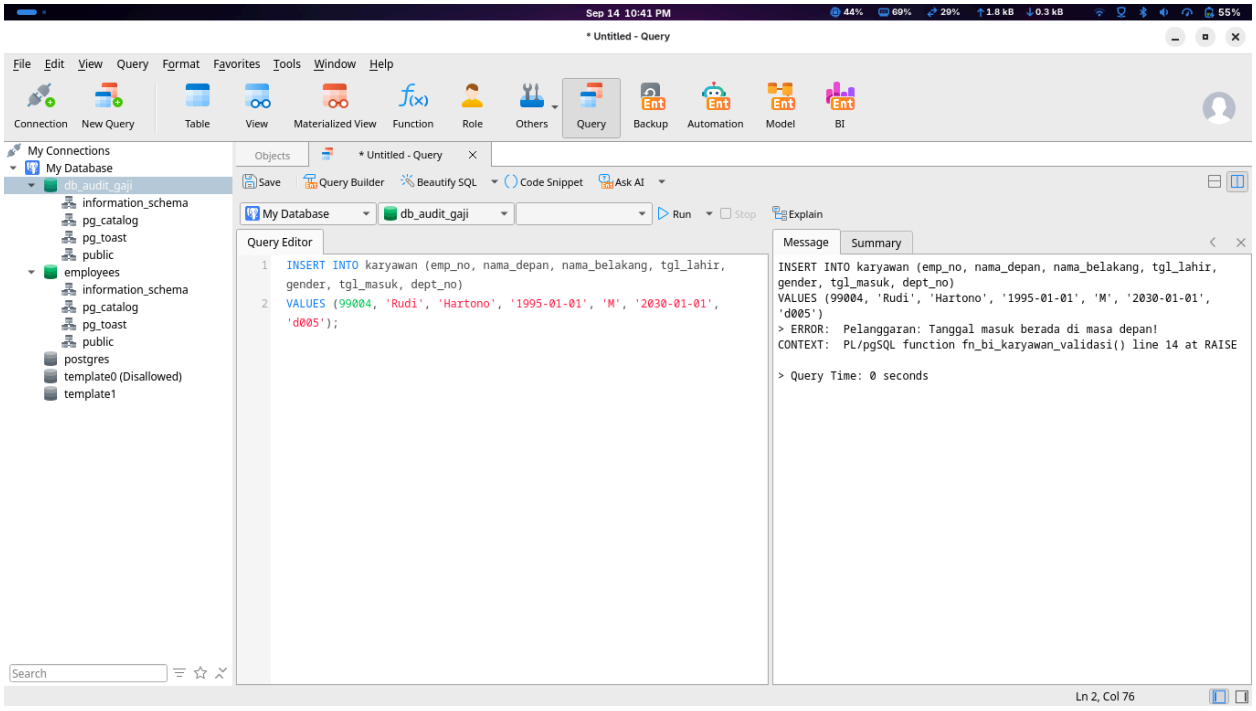
Tahap B.3 - Membuat Trigger Validasi Data (B3)

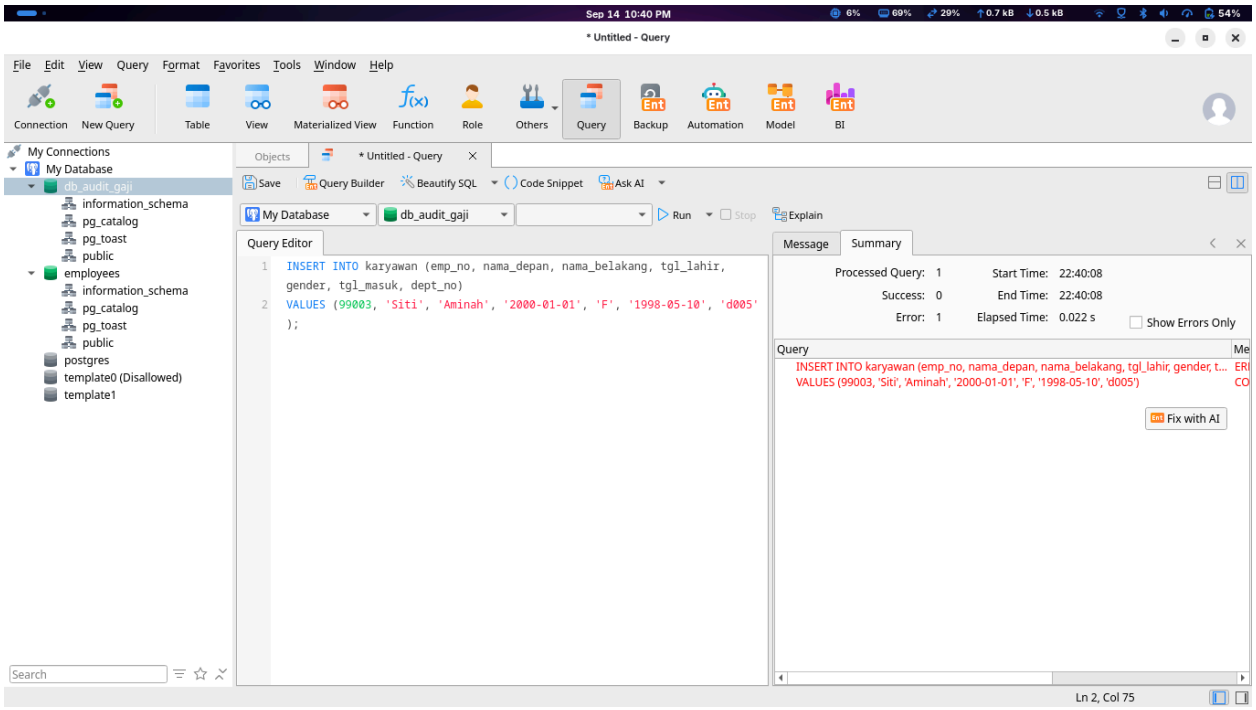
Membuat aturan agar sistem otomatis menolak data karyawan di bawah umur 18 tahun atau tanggal yang salah.

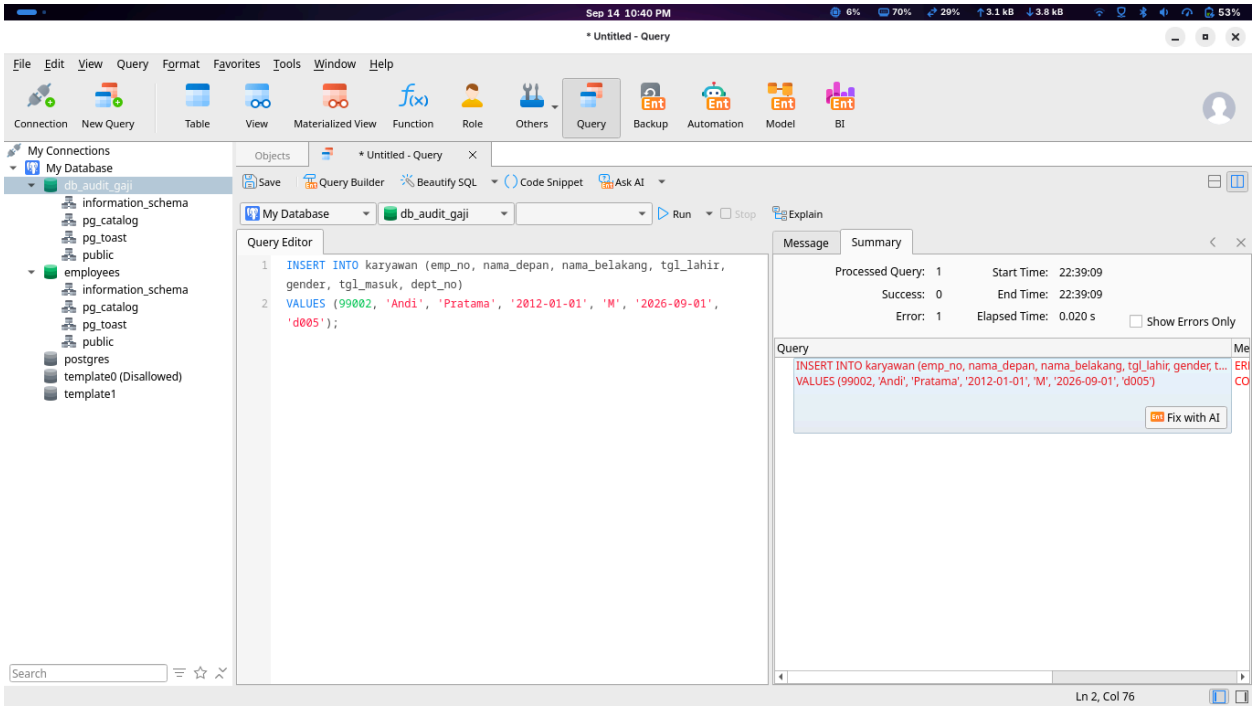


Tahap B.4 - Uji 3 Kondisi Validasi (B4)

Menguji coba memasukkan data yang melanggar batas usia dan tanggal untuk melihat pesan error-nya.







Keunggulan validasi di basis data:

1. Data lebih aman dan konsisten karena aturan langsung dicegat di level database, jadi aplikasi atau query apa pun tidak bisa memasukkan data sembarangan.
2. Mengurangi duplikasi kode di backend, jadi tidak perlu membuat pengecekan berulang di tiap endpoint.

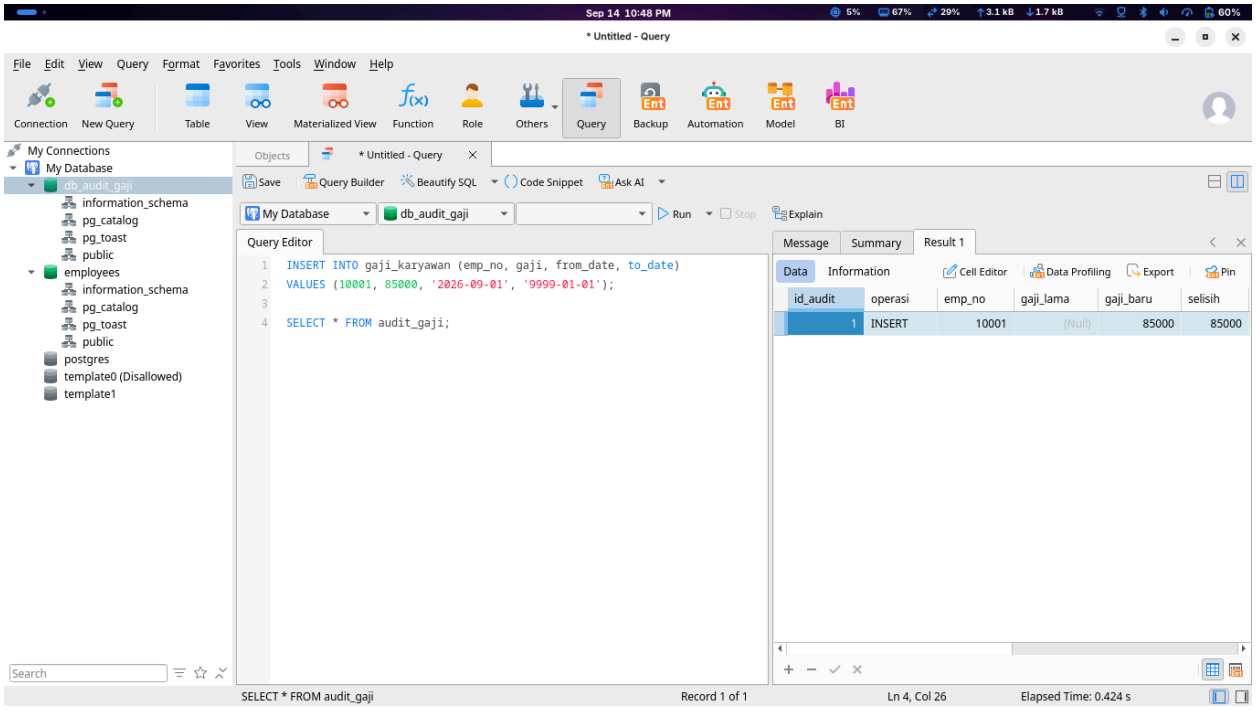
Kekurangan validasi di basis data:

1. Menambah beban kerja CPU server database jika transaksi masuk dalam jumlah besar sekaligus.
2. Pesan error dari database lebih rumit diproses oleh backend sebelum ditampilkan ke tampilan pengguna, serta sintaksnya terikat pada dialek PostgreSQL.

Bagian C - Trigger Jejak Audit

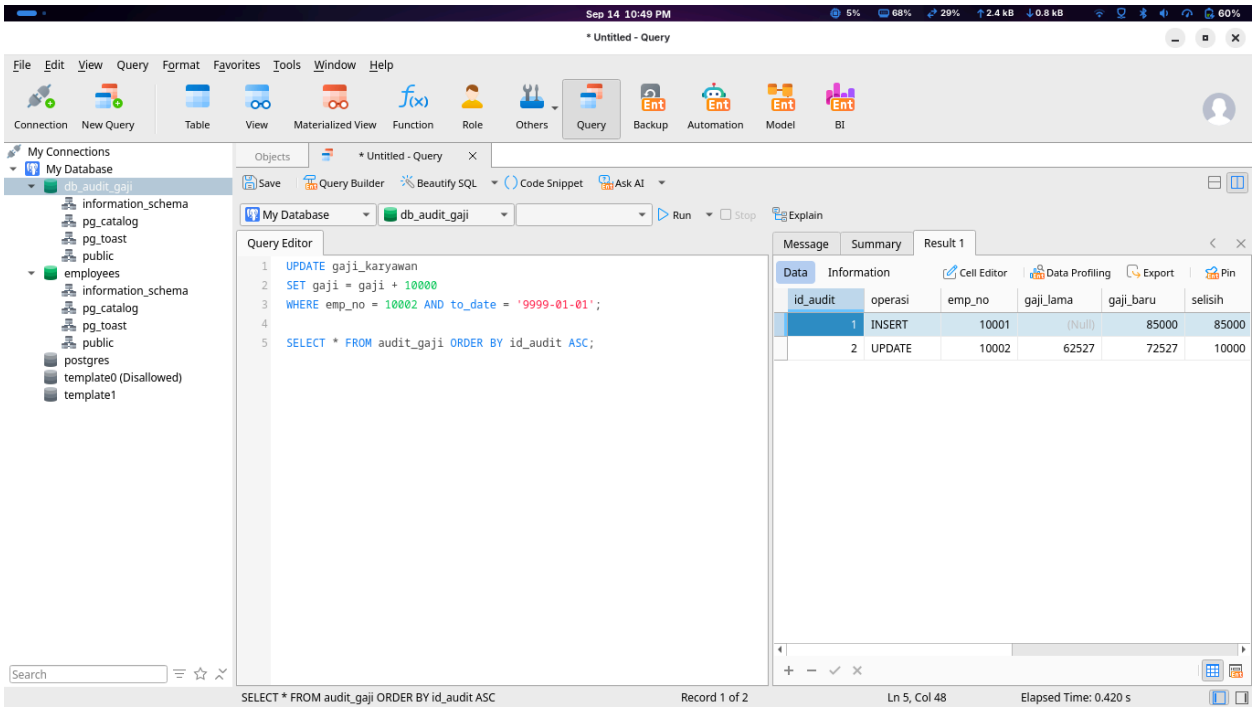
Langkah C.4.1 - Uji Transaksi INSERT (Menyisipkan Baris Gaji Baru)

Menguji apakah penambahan gaji baru berhasil tercatat otomatis ke tabel audit.



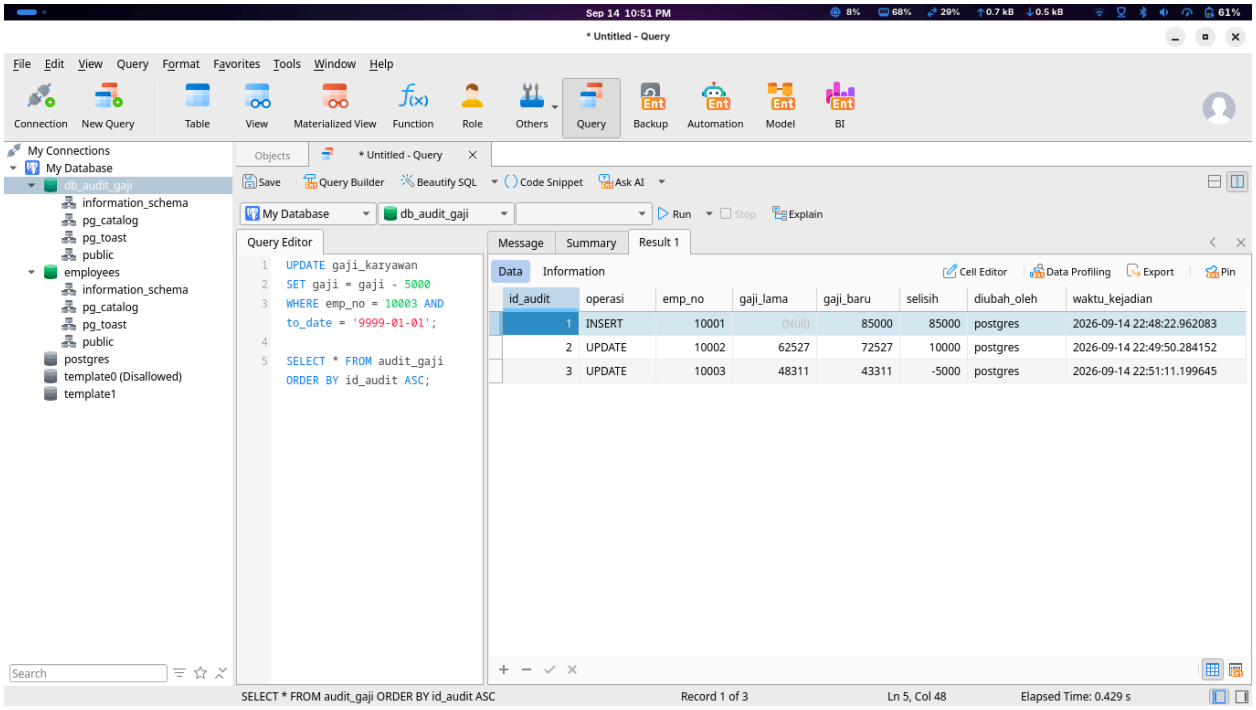
Langkah C.4.2 - Uji Transaksi UPDATE Naik

Memastikan kenaikan nominal gaji beserta jumlah selisihnya tercatat di tabel audit.

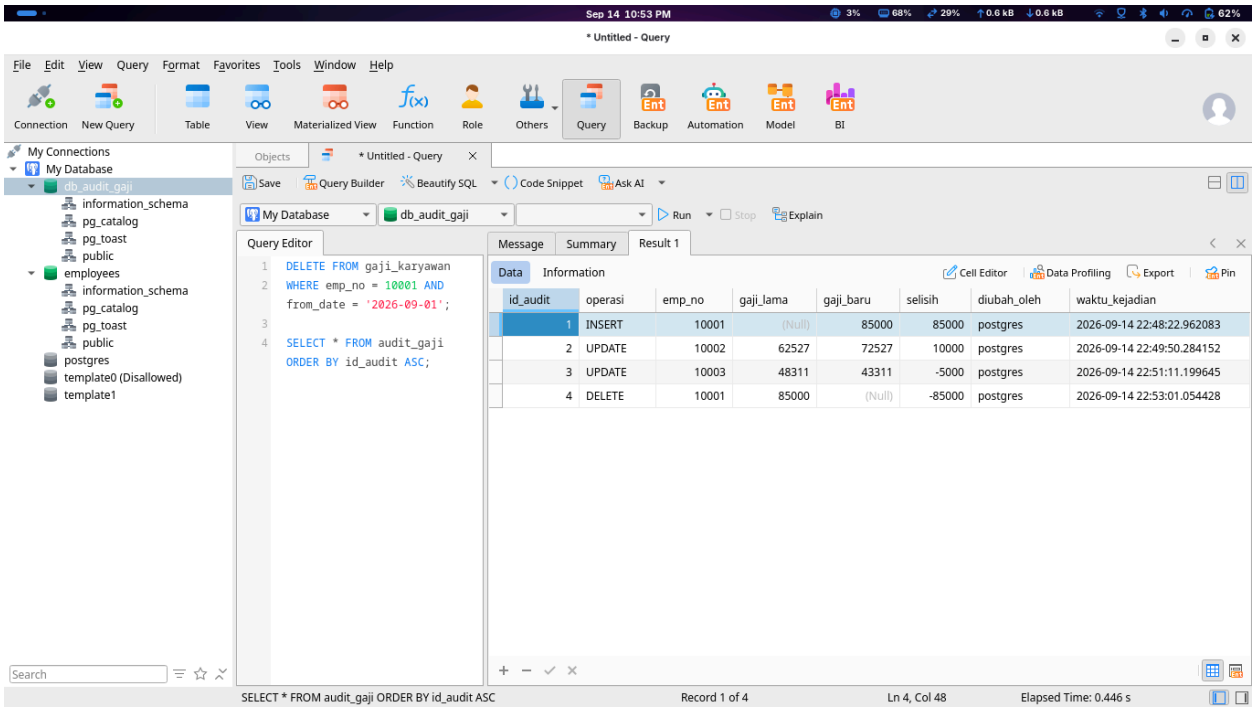


Langkah C.4.3 - Uji Transaksi UPDATE Turun

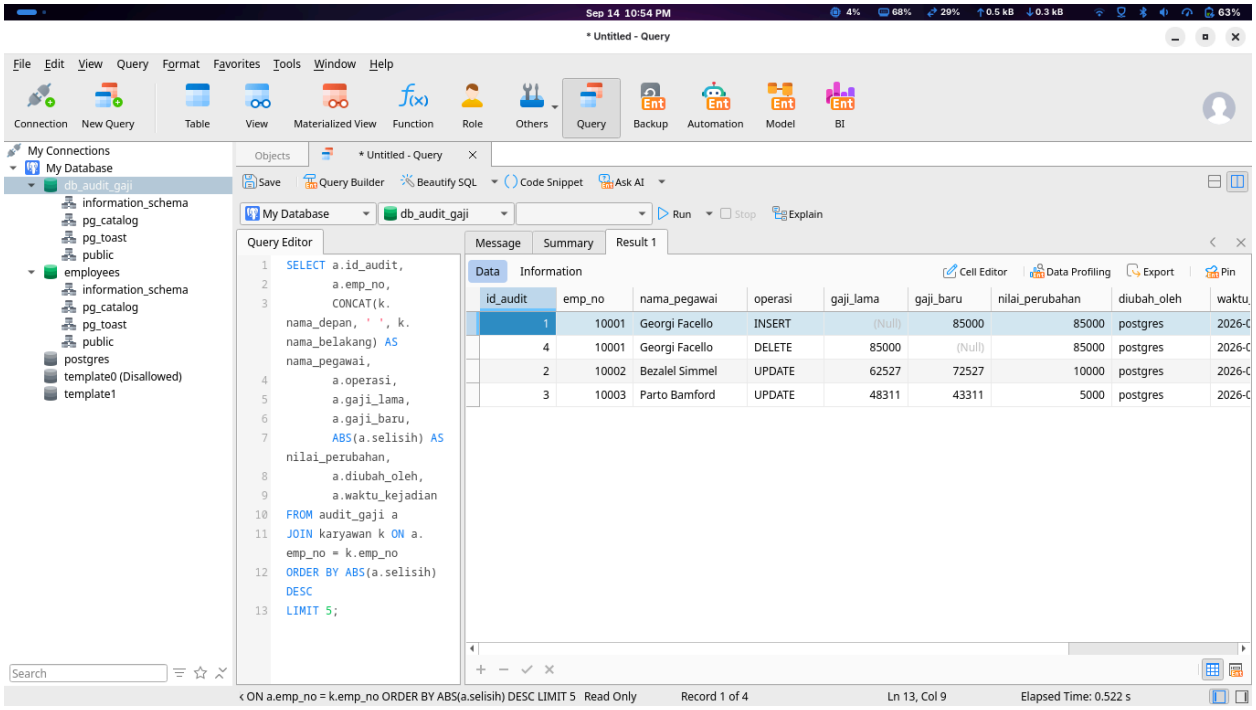
Memastikan penurunan nominal gaji juga ikut terekam lengkap di tabel audit.



Langkah C.4.4 - Uji Transaksi DELETE (Menghapus Baris Gaji)
Membuktikan data gaji yang dihapus tetap tersimpan jejaknya di tabel audit.



Tahap C.5 - Query Top Perubahan Gaji Terbesar
Menampilkan lima data dengan perubahan nominal gaji paling drastis dari tabel audit.



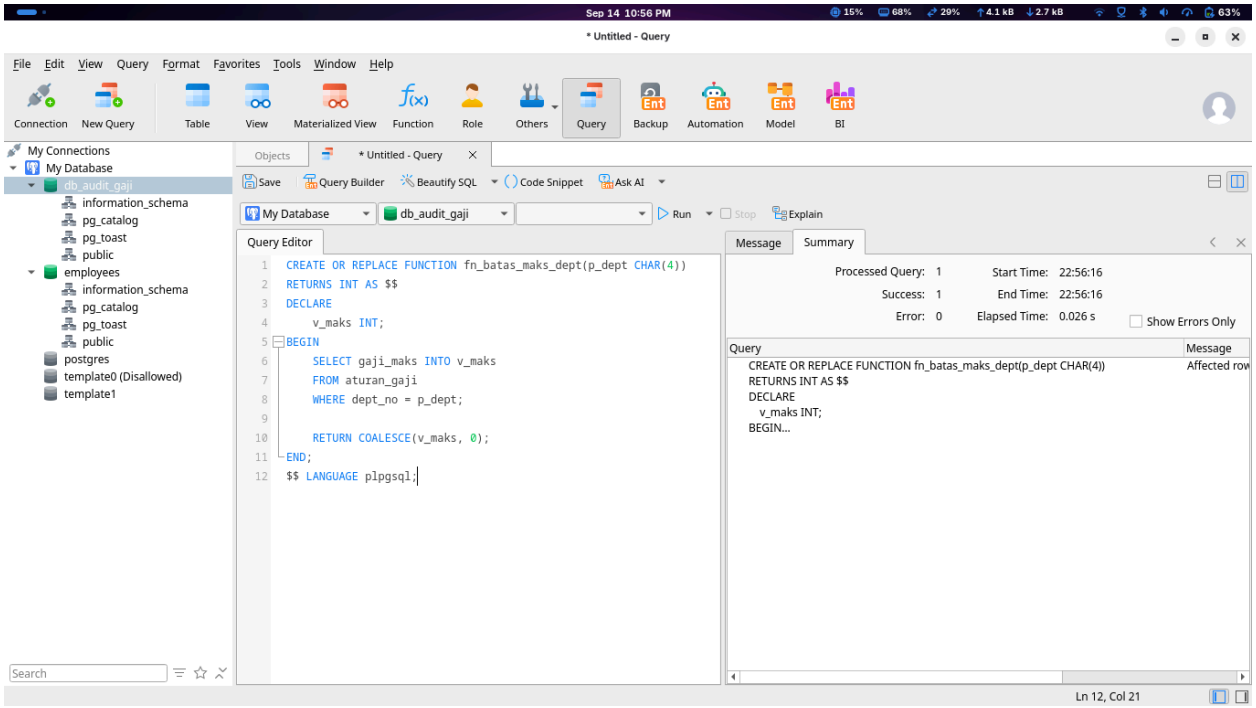
Mengapa menggunakan AFTER dan bukan BEFORE? Trigger audit memakai AFTER untuk memastikan bahwa perubahan data pada tabel gaji_karyawan benar-benar sudah berhasil diproses dan disimpan ke tabel.

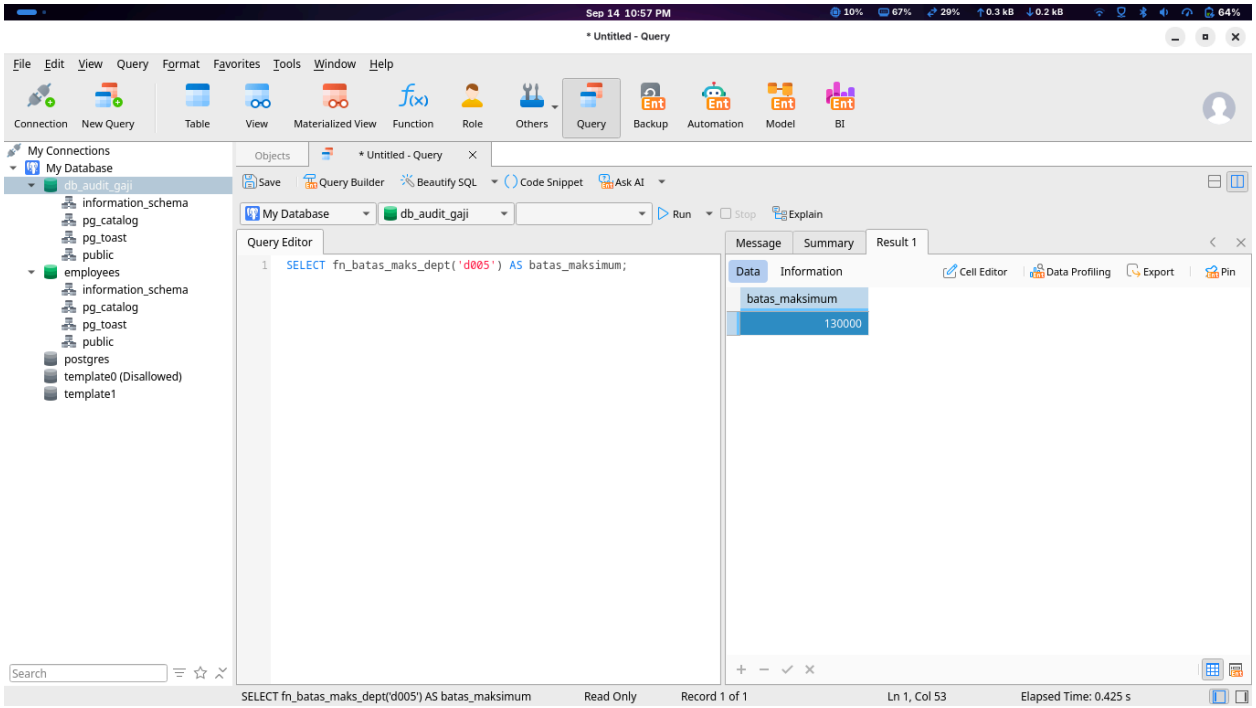
Apa yang terjadi jika menggunakan BEFORE dan operasinya gagal? Jika memakai BEFORE, riwayat audit akan tercatat lebih dulu sebelum transaksi utama selesai. Jika setelah itu eksekusi gajinya gagal atau error, tabel audit akan mencatat riwayat perubahan yang sebenarnya tidak pernah tersimpan di tabel utama.

Bagian D - Trigger Penegakan Aturan Bisnis

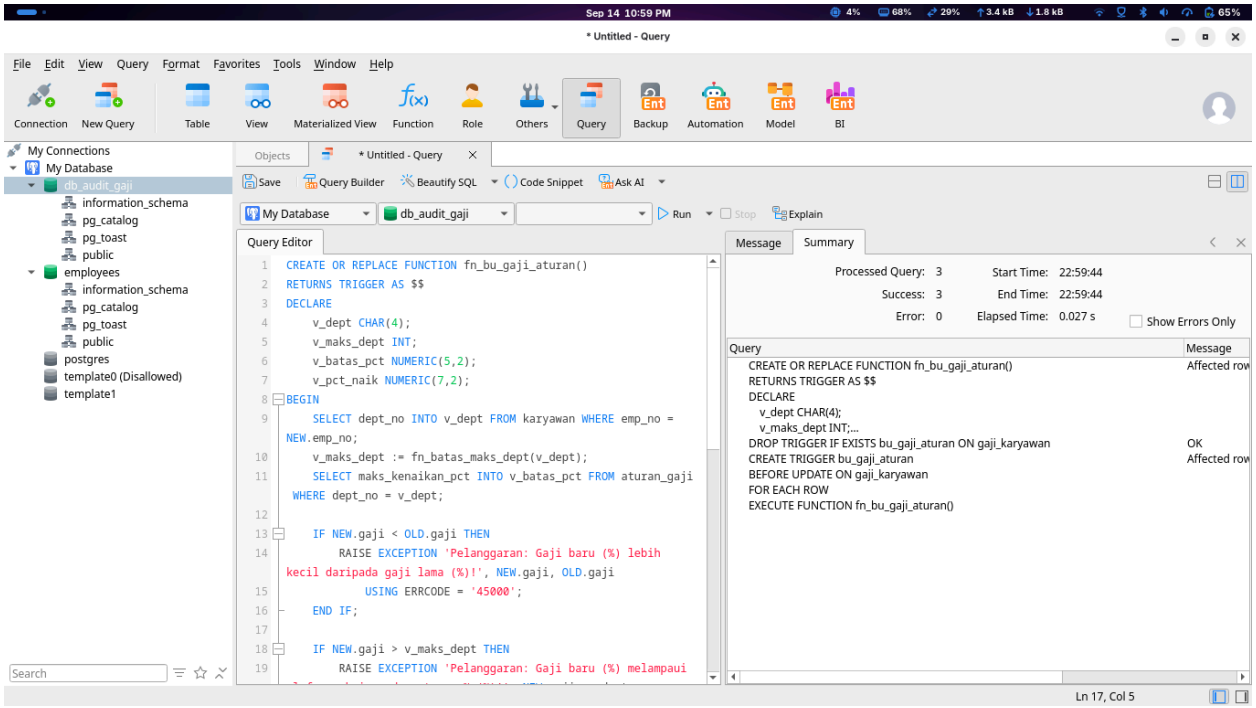
Tahap D.1 - Fungsi Pembantu Batas Maksimum

Membuat fungsi kecil untuk mengecek gaji tertinggi yang dibolehkan di suatu departemen.



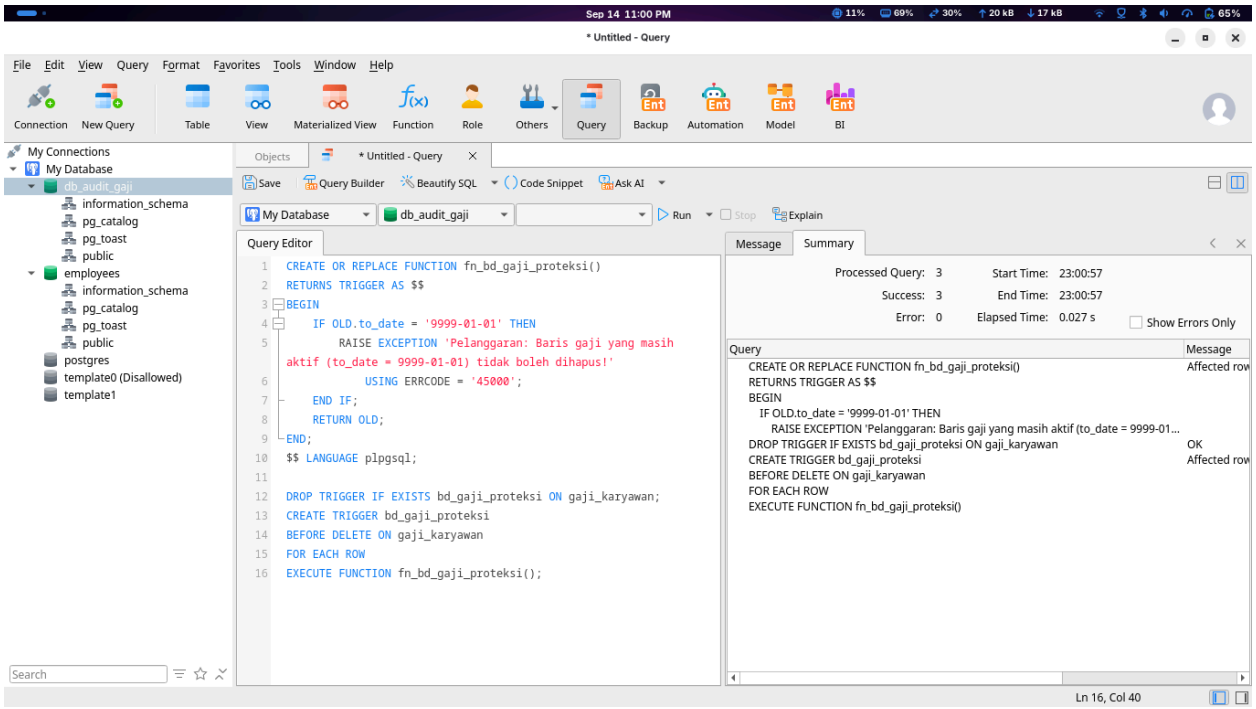


Tahap D.2 - Membuat Trigger Validasi Aturan Gaji (bu_gaji_aturan)
Memasang cegatan agar gaji tidak boleh turun, kenaikan tidak lewat persentase, dan tidak lewat batas departemen.



Tahap D.3 - Membuat Trigger Proteksi Hapus (bd_gaji_proteksi)

Mencegah penghapusan riwayat gaji karyawan yang statusnya masih berlaku.

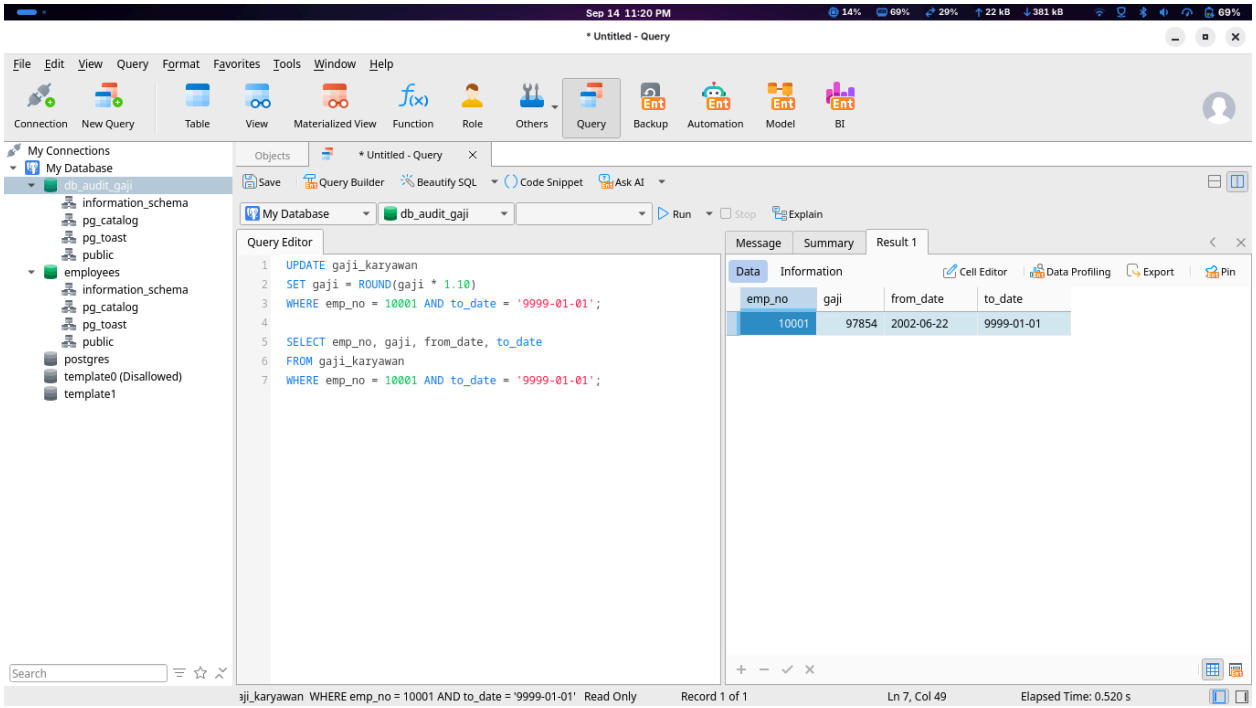


Tahap D.4 - Rangkaian Pengujian Aturan Bisnis (6 Kasus Uji)

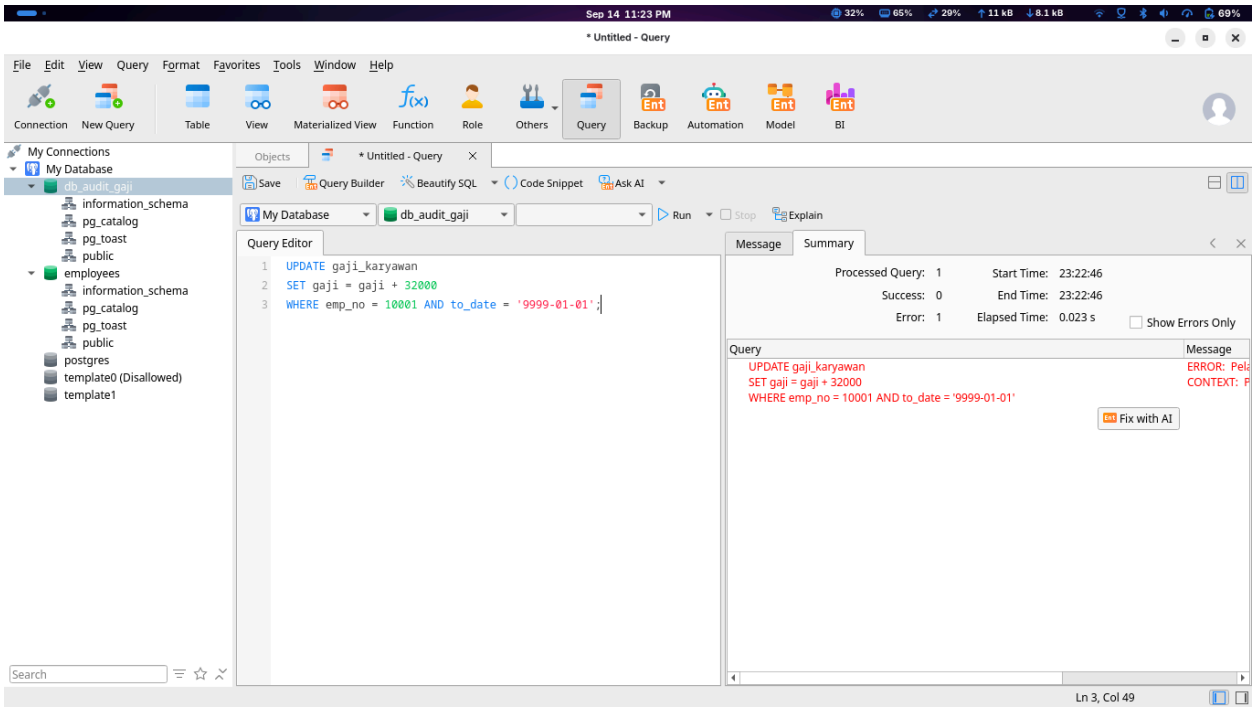
Menguji 6 skenario perubahan gaji untuk memastikan sistem menolak update yang melanggar

aturan.

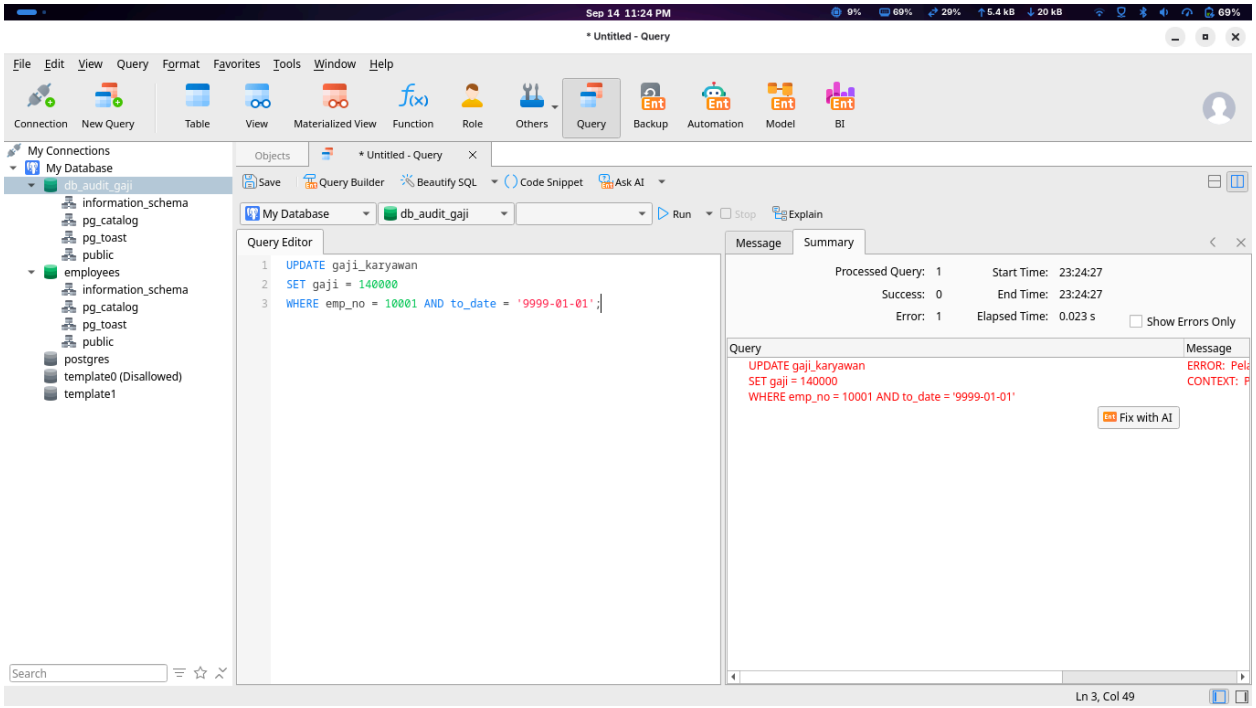
Kasus Uji 1: Menaikkan Gaji 10% (Valid / Berhasil)



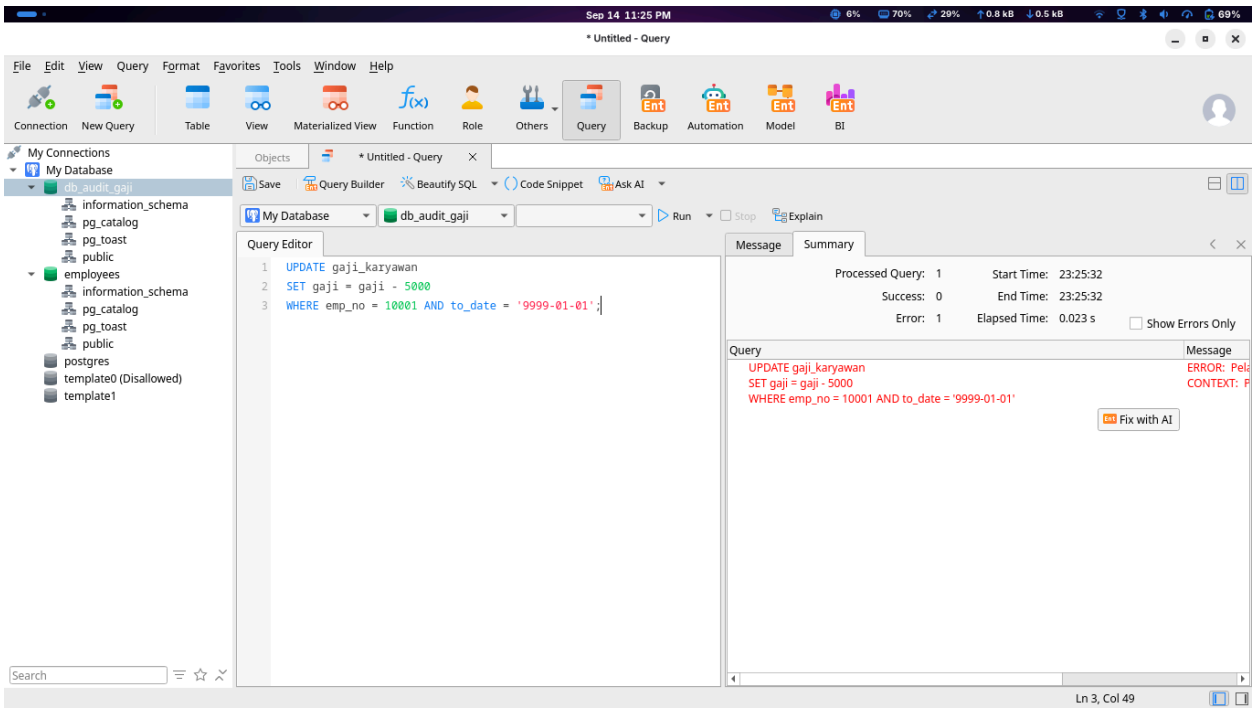
Kasus Uji 2: Menaikkan Gaji 50% (Melampaui Maksimal Kenaikan 30%)



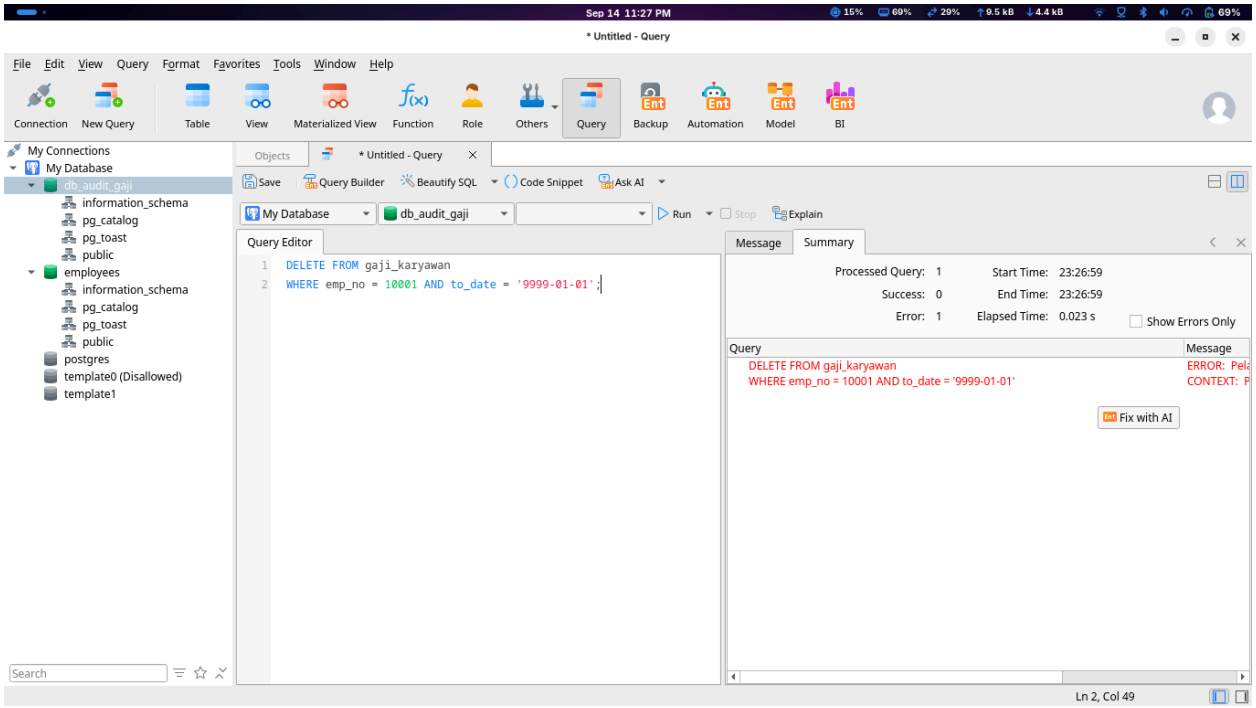
Kasus Uji 3: Menaikkan Gaji Melampaui Batas Maksimum Departemen



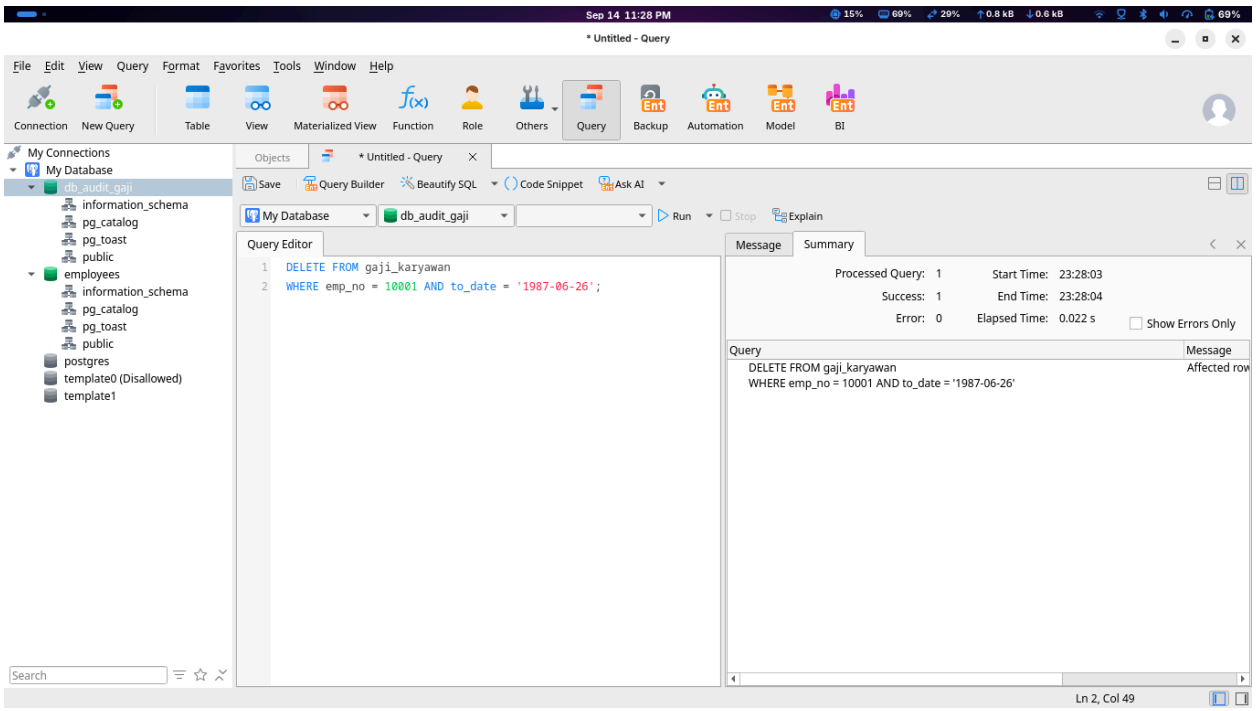
Kasus Uji 4: Menurunkan Gaji (Pelanggaran Aturan Gaji Turun)



Kasus Uji 5: Menghapus Baris Gaji yang Masih Aktif (Pelanggaran Proteksi Hapus)

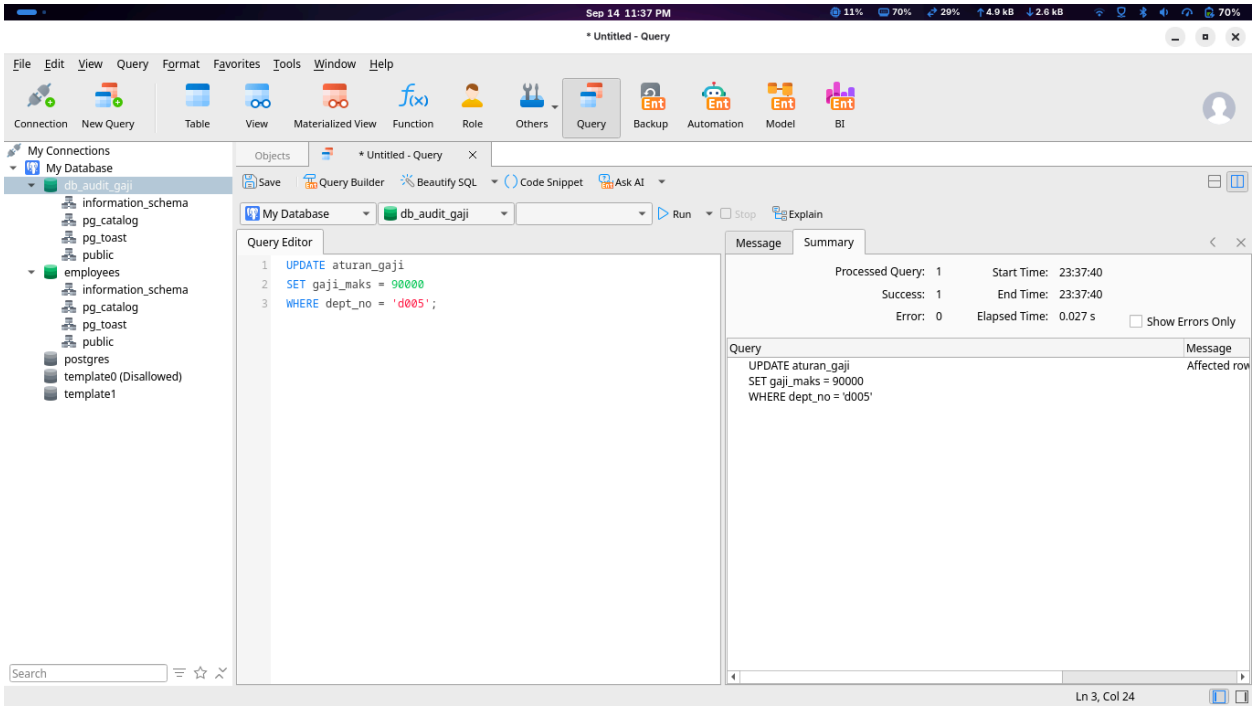


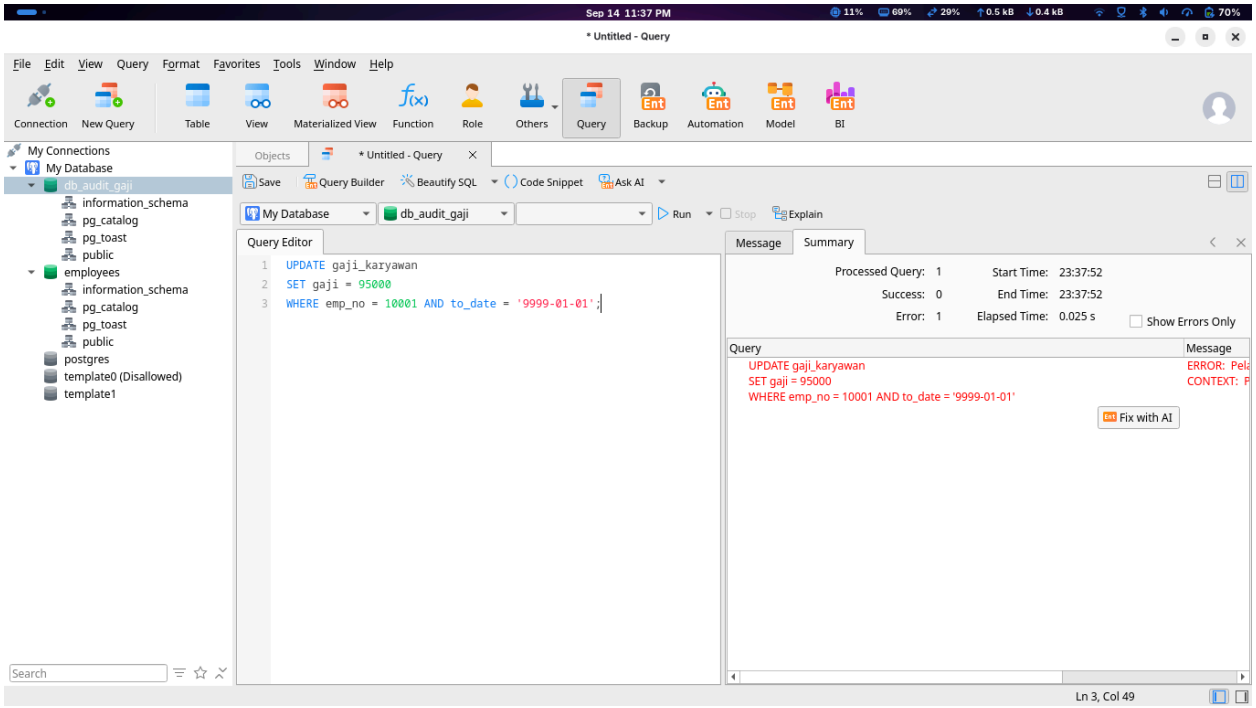
Kasus Uji 6: Menghapus Baris Gaji Riwayat Lama (Valid / Berhasil)



Tahap D.5 - Menguji Parameter Aturan yang Diubah Dinamis

Membuktikan aturan gaji langsung menyesuaikan otomatis jika batas di tabel aturan kita ubah.





Mengapa validasi diletakkan di BEFORE UPDATE?
Agar query bisa langsung dibatalkan sebelum data sempat tersimpan ke database, jadi sistem tidak perlu repot melakukan rollback atau memicu trigger audit.

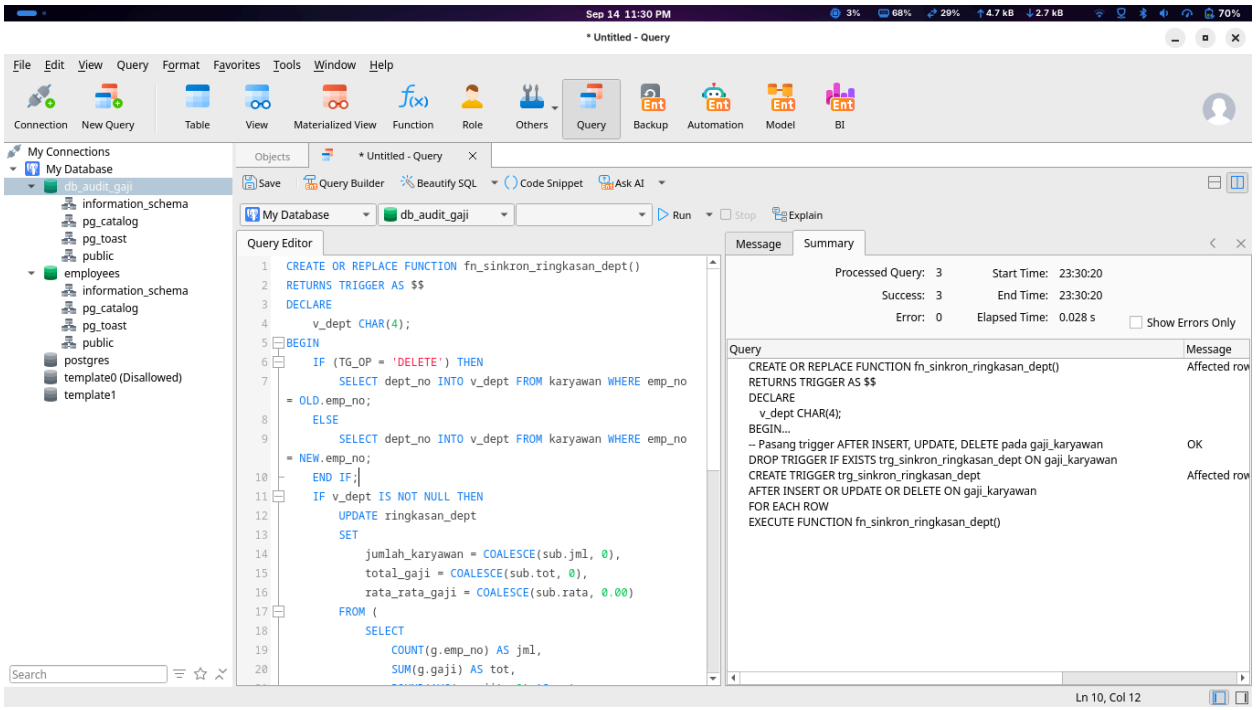
Dampaknya jika dijalankan pada update data sekaligus banyak:

Karena trigger mengecek baris satu per satu, kueri ke tabel aturan akan dieksekusi terus-menerus sebanyak data yang diubah, sehingga proses update massal menjadi lebih lambat.

Bagian E - Pemeliharaan Ringkasan Otomatis (Maintenance Aggregate Table)

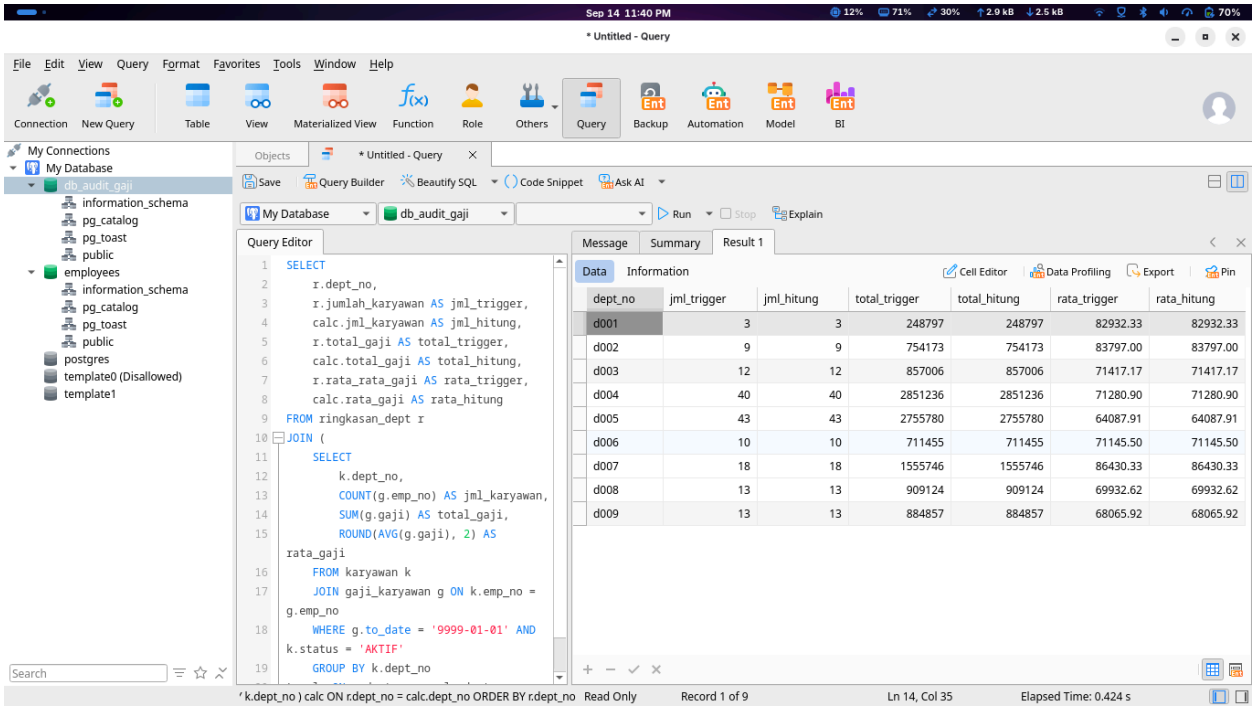
Tahap E.1 - Membuat Trigger Fungsi Sinkronisasi Agregat Departemen

Membuat ringkasan total dan rata-rata gaji departemen ter-update sendiri saat ada transaksi.



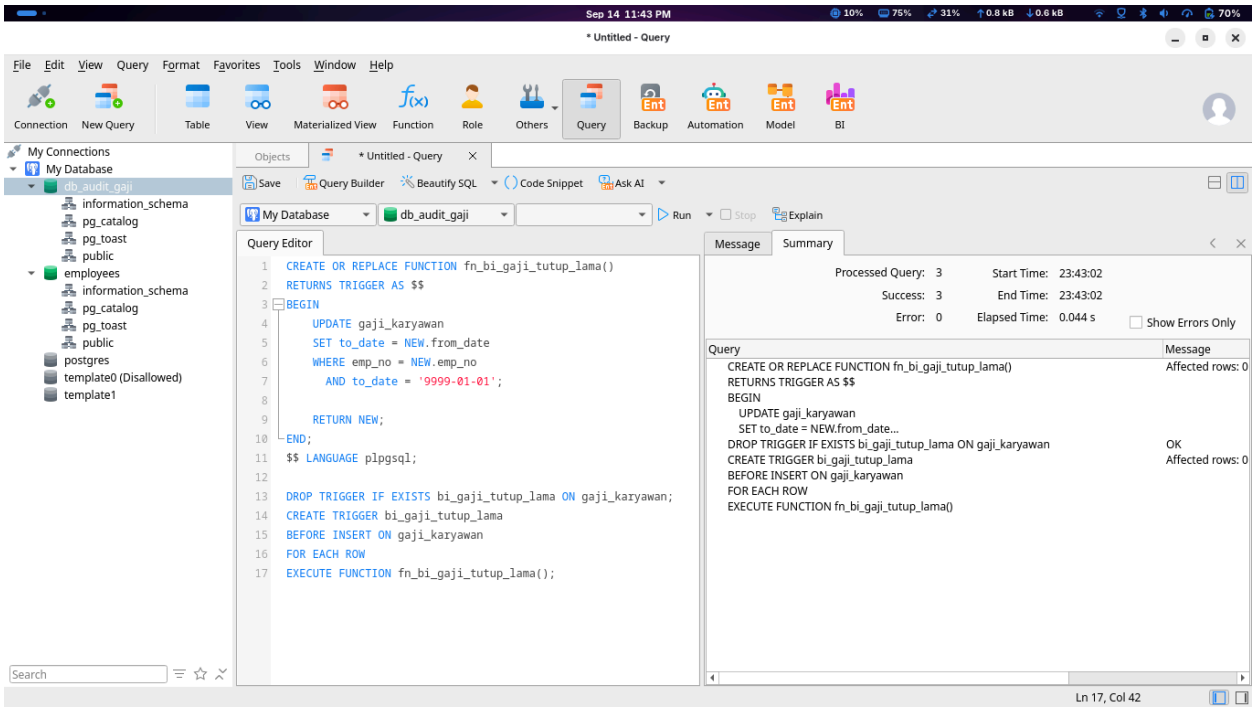
Tahap E.2 - Verifikasi Ketepatan Agregat (Perbandingan Langsung)

Mencocokkan angka di tabel ringkasan otomatis dengan hitungan manual agar terbukti sama.



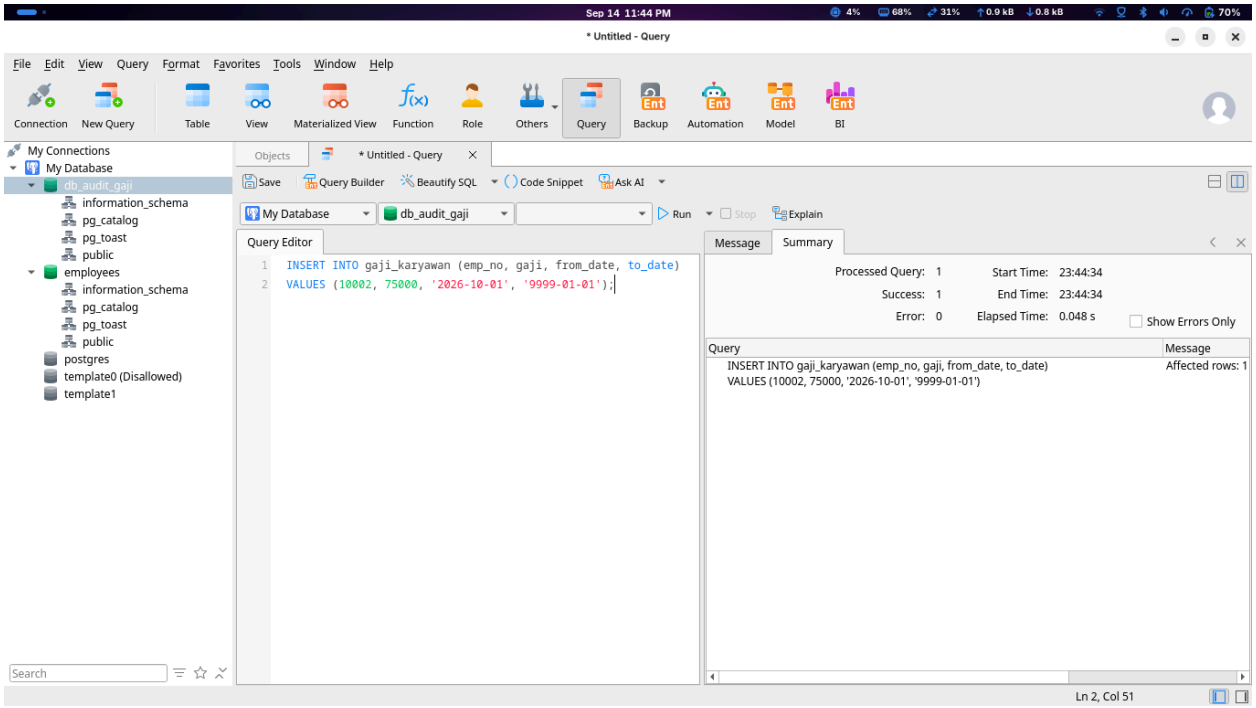
Tahap E.3 - Membuat Trigger bi_gaji_tutup_lama

Mencoba menutup tanggal gaji lama secara otomatis tepat saat ada data gaji baru yang masuk.



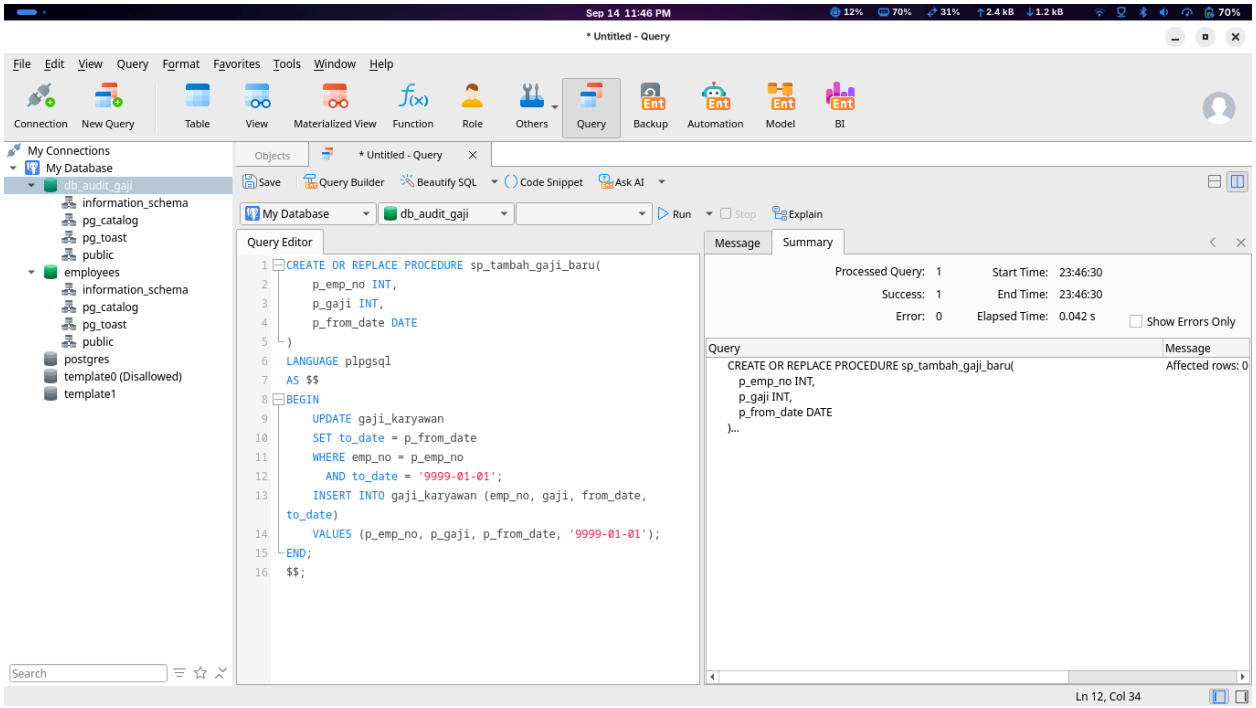
Langkah E.4 - Pengujian dan Analisis Keterbatasan Trigger

Melihat efek dan risiko saat trigger mencoba mengubah data pada tabel yang sedang diproses.



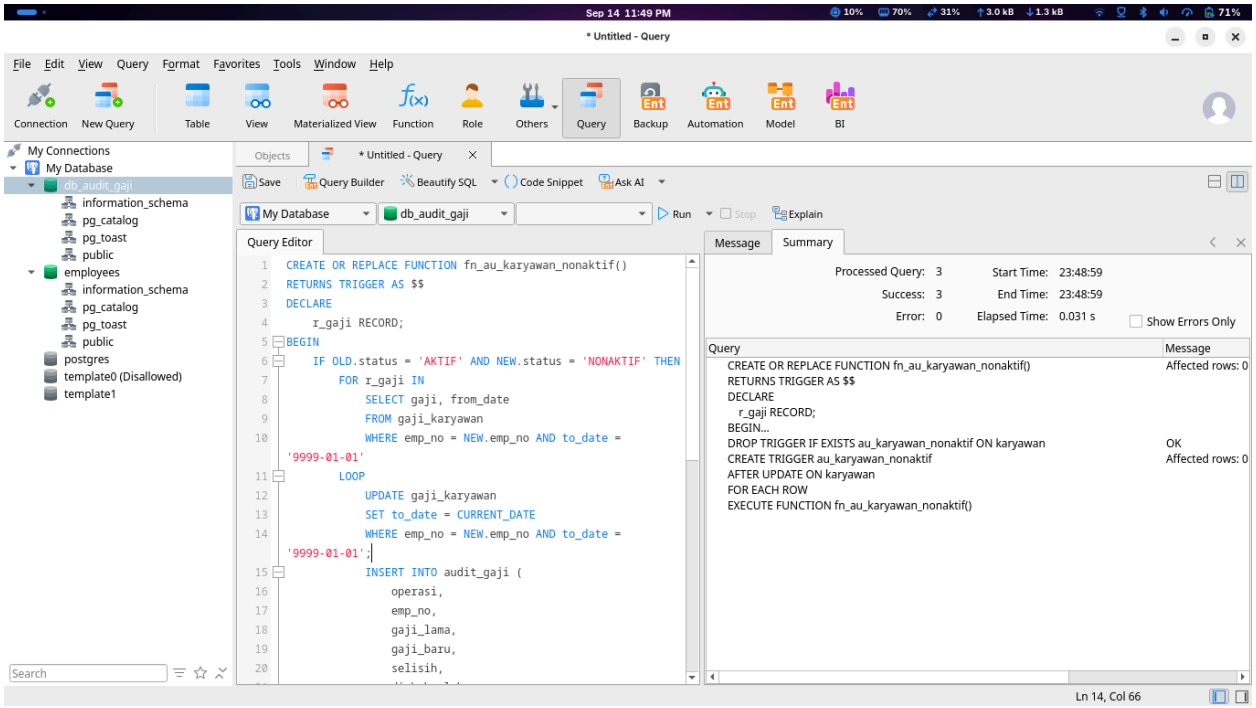
Kalau di MySQL atau MariaDB, skenario ini langsung memicu galat *mutating table* (Error 1442) karena database melarang trigger mengubah tabel yang sedang menjadi target operasi utama. Sedangkan di PostgreSQL operasi ini tetap dibolehkan, tetapi berbahaya karena bisa memicu trigger berulang tanpa henti (*recursive trigger*) atau bentrok data jika ada proses lain yang berjalan bersamaan.

Tahap E.5 - Solusi Alternatif (Stored Procedure / Transaksi Terkelola)
Memakai stored procedure sebagai jalan keluar yang lebih aman untuk mengganti data gaji.

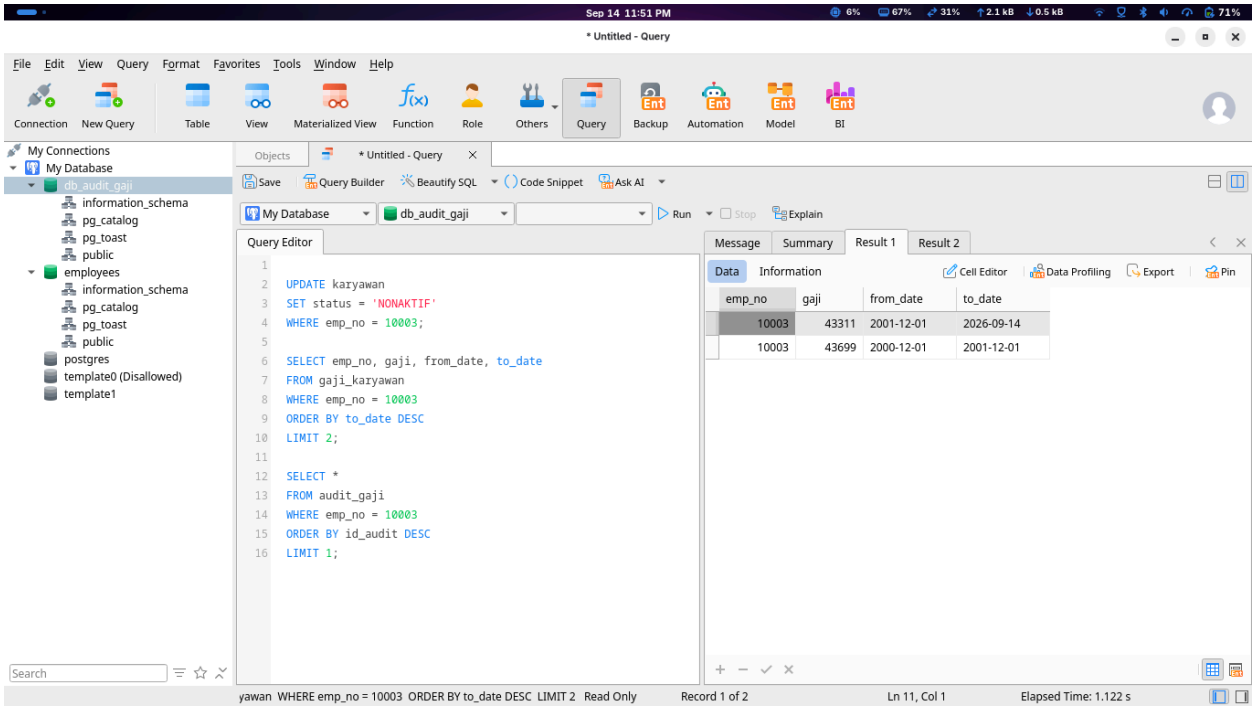


Tahap E.6 - Membuat Trigger Penonaktifan Karyawan (au_karyawan_nonaktif)

Menutup masa aktif gaji secara otomatis saat seorang karyawan dinonaktifkan.



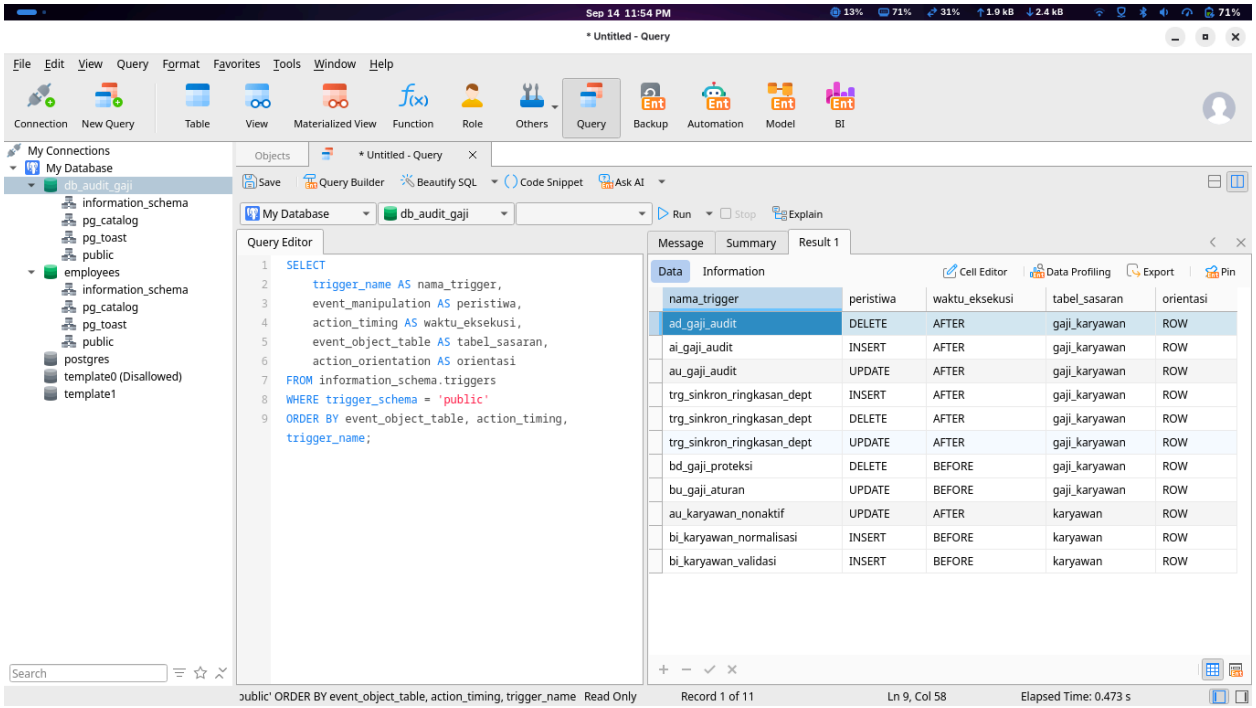
Uji Coba Langkah E.6 - Mengubah Status Pegawai Menjadi NONAKTIF



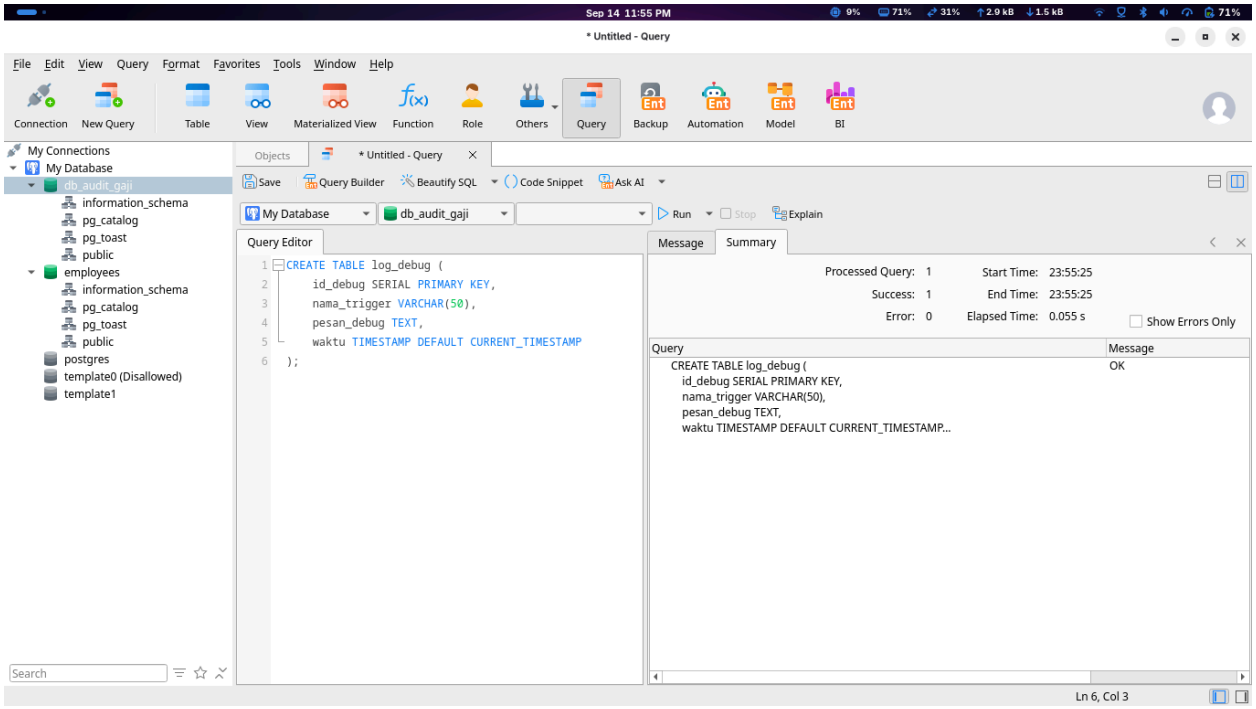
Bagian F - Pengujian, Penelusuran, dan Pengelolaan Trigger

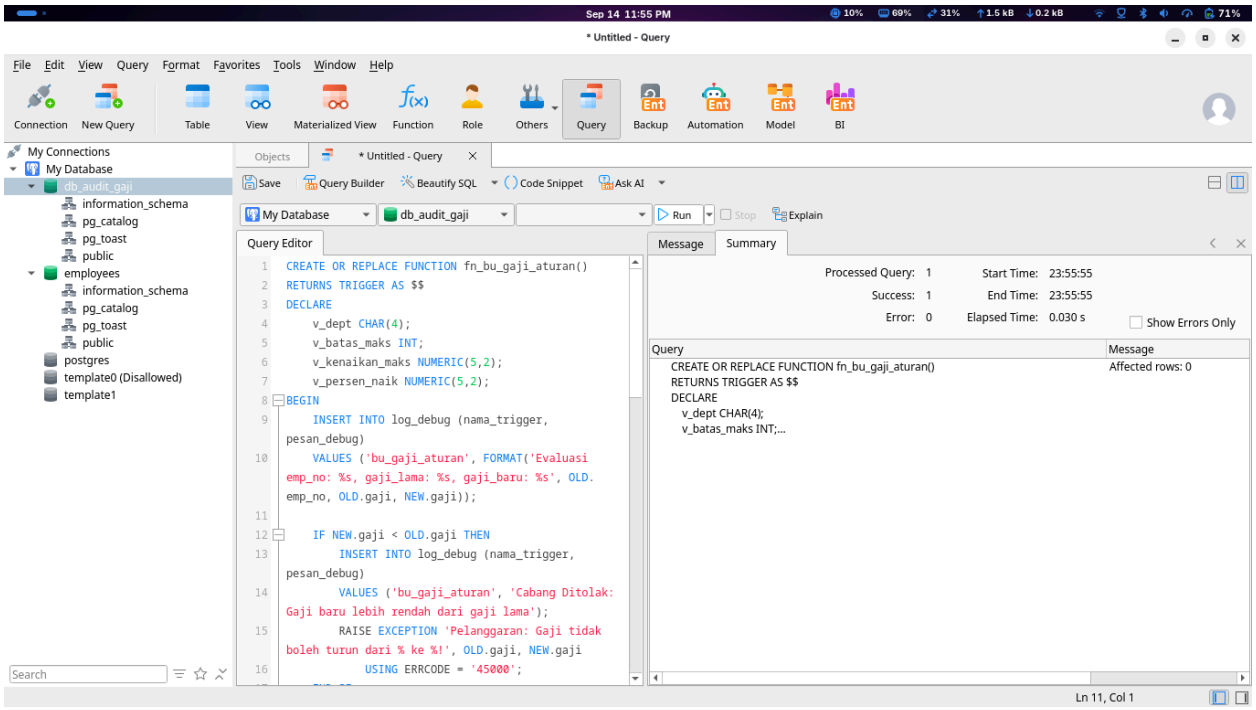
Tahap F.1 - Menampilkan Seluruh Inventaris Trigger

Melihat daftar semua trigger yang sudah aktif dipasang di dalam database.

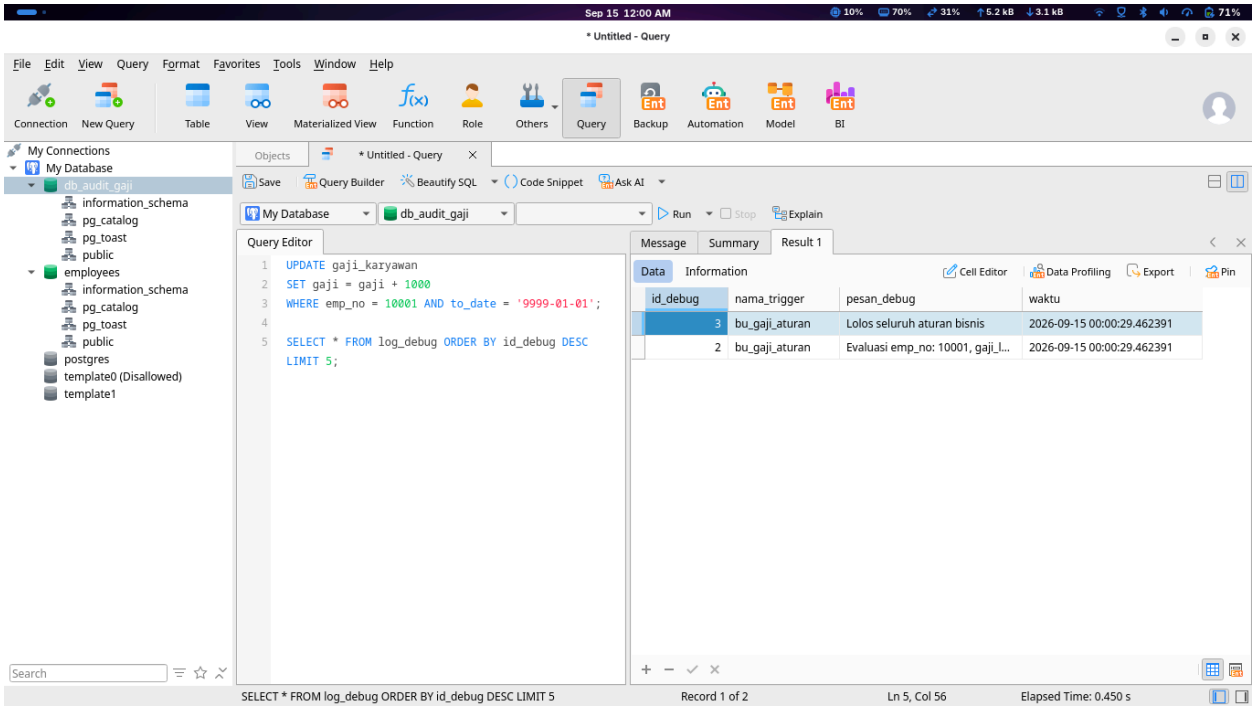


Tahap F.2 - Perancangan Tabel log_debug dan Penelusuran Percabangan Logika
Menggunakan tabel log khusus untuk melacak jalan pikiran trigger saat memproses query.

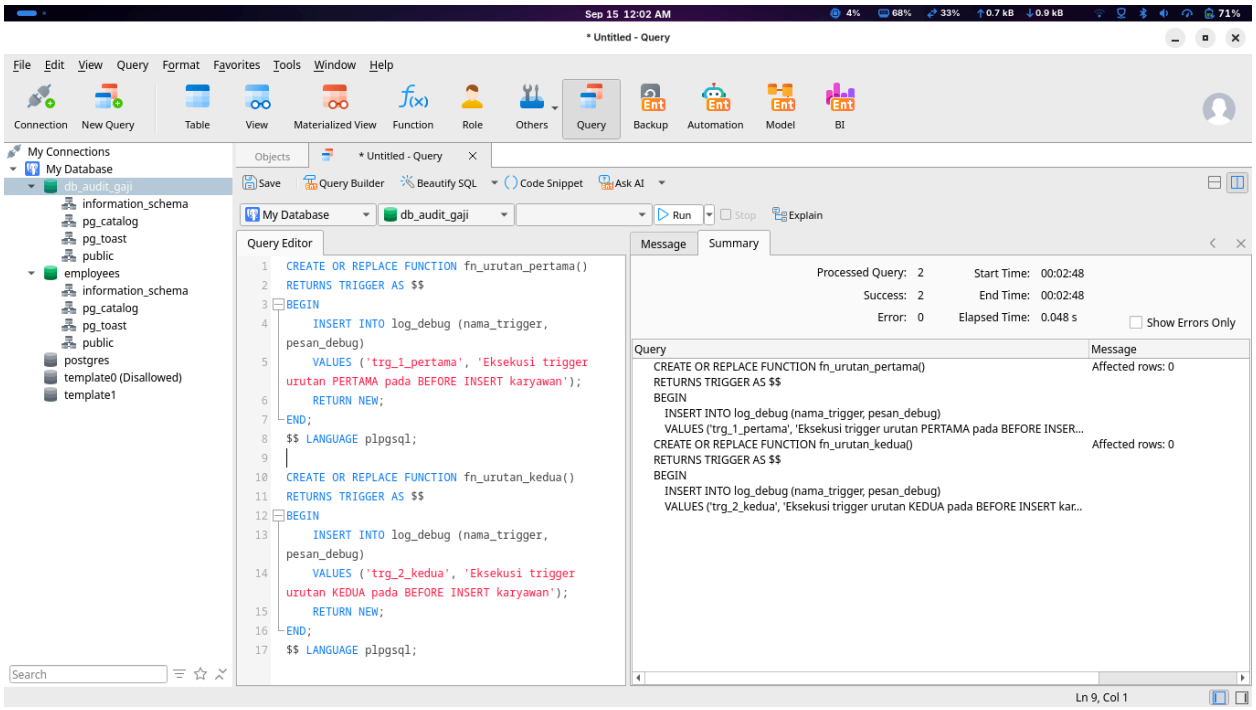




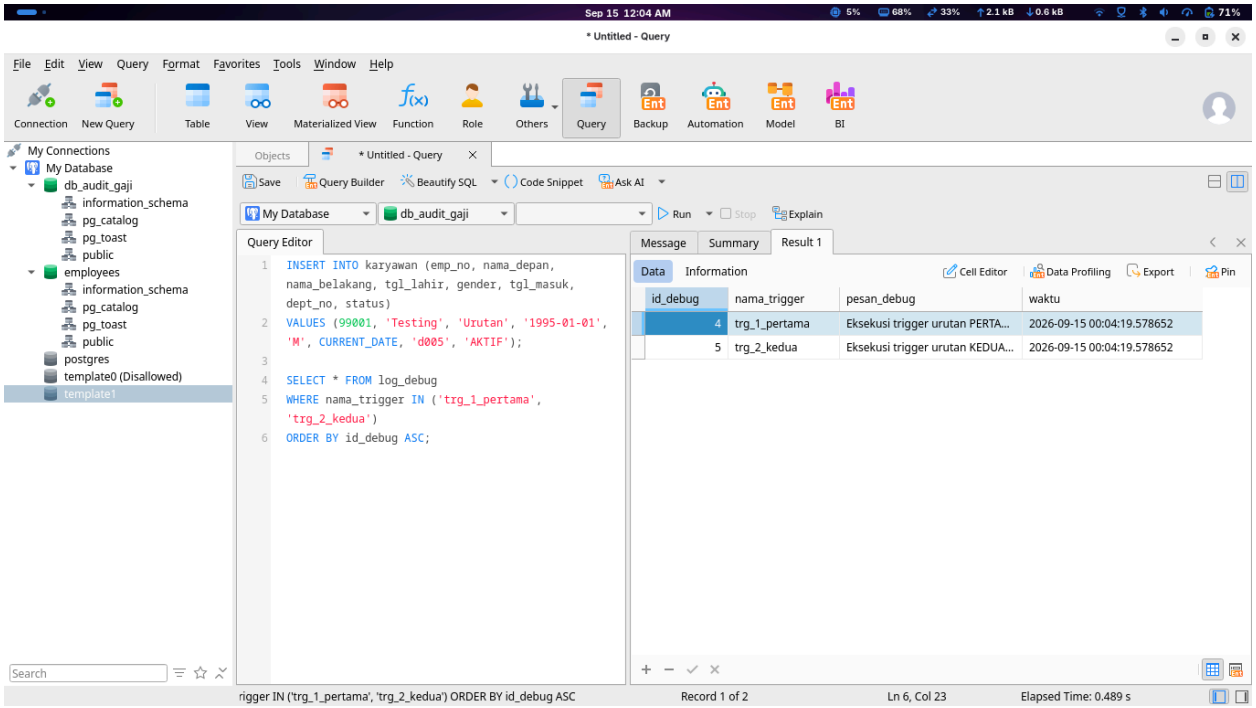
Pengujian Tahap F.2 - Penelusuran Percabangan Logika



Tahap F.3 - Pengurutan Eksekusi Trigger pada Tabel yang Sama
Membuktikan trigger berjalan berurutan sesuai abjad namanya.

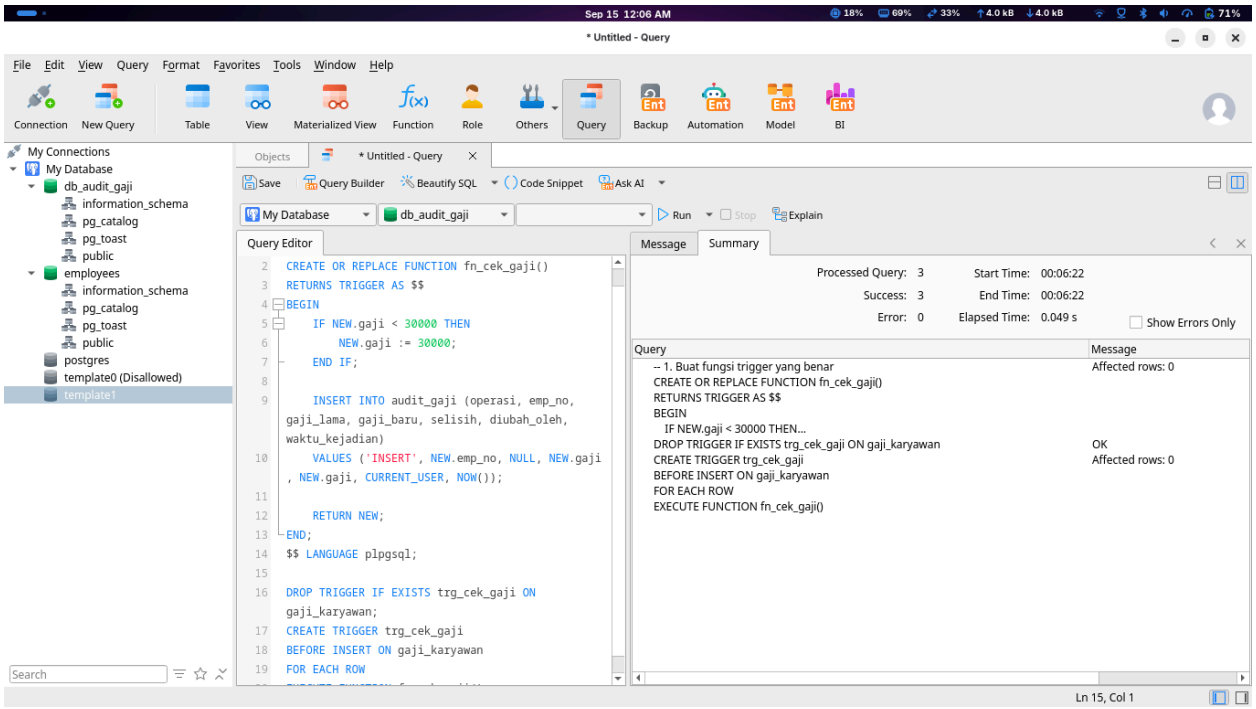


F.3 - Hasil Uji Urutan Eksekusi.



Tahap F.4 - Studi Kasus Penelusuran Galat

Menemukan dan membetulkan salah ketik penempatan variabel trigger pada contoh kasus error.



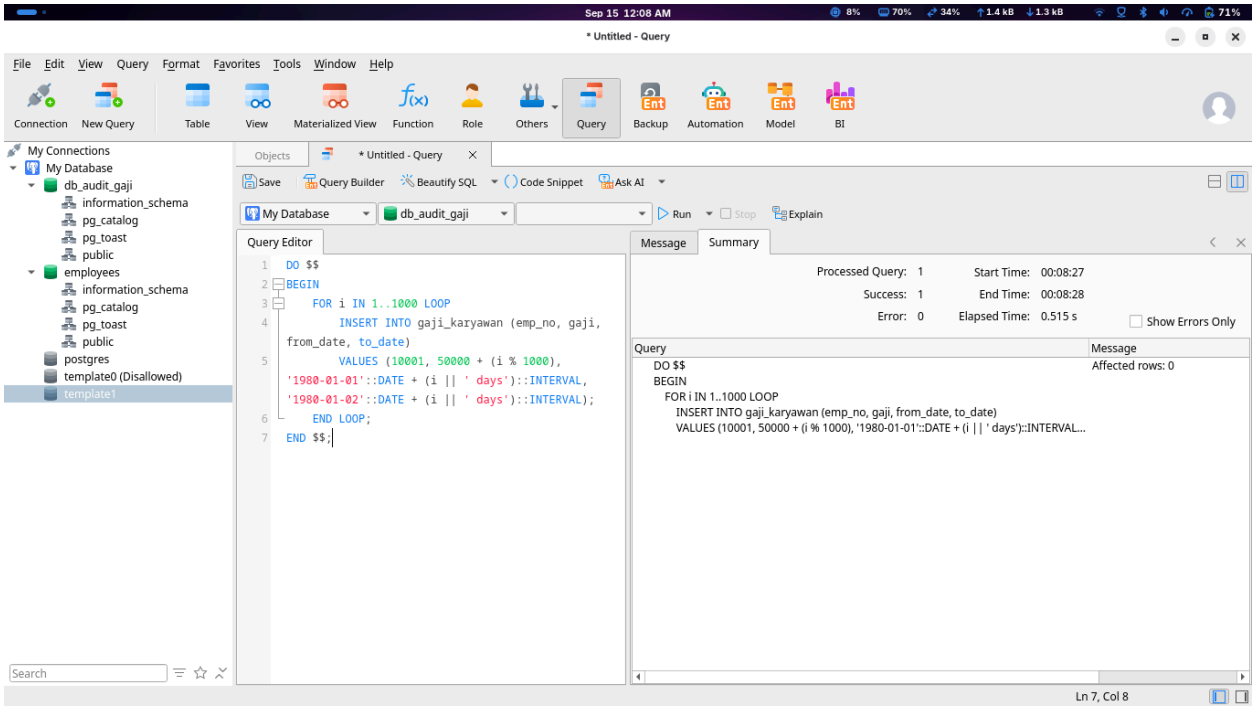
Temuan Kesalahan dan Analisis:

1. Modifikasi NEW pada trigger AFTER INSERT: Pernyataan SET NEW.gaji = 30000; tidak sah karena pada trigger AFTER, data sudah tersimpan permanen ke disk dan record NEW bersifat *read-only*. Pengubahan nilai wajib menggunakan BEFORE INSERT.
2. Penggunaan record OLD pada peristiwa INSERT: Pernyataan INSERT INTO audit_gaji ... VALUES (OLD.emp_no, ...) keliru karena aksi penyisipan baris baru tidak memiliki data sebelumnya (OLD bernilai NULL). Seharusnya menggunakan NEW.emp_no.

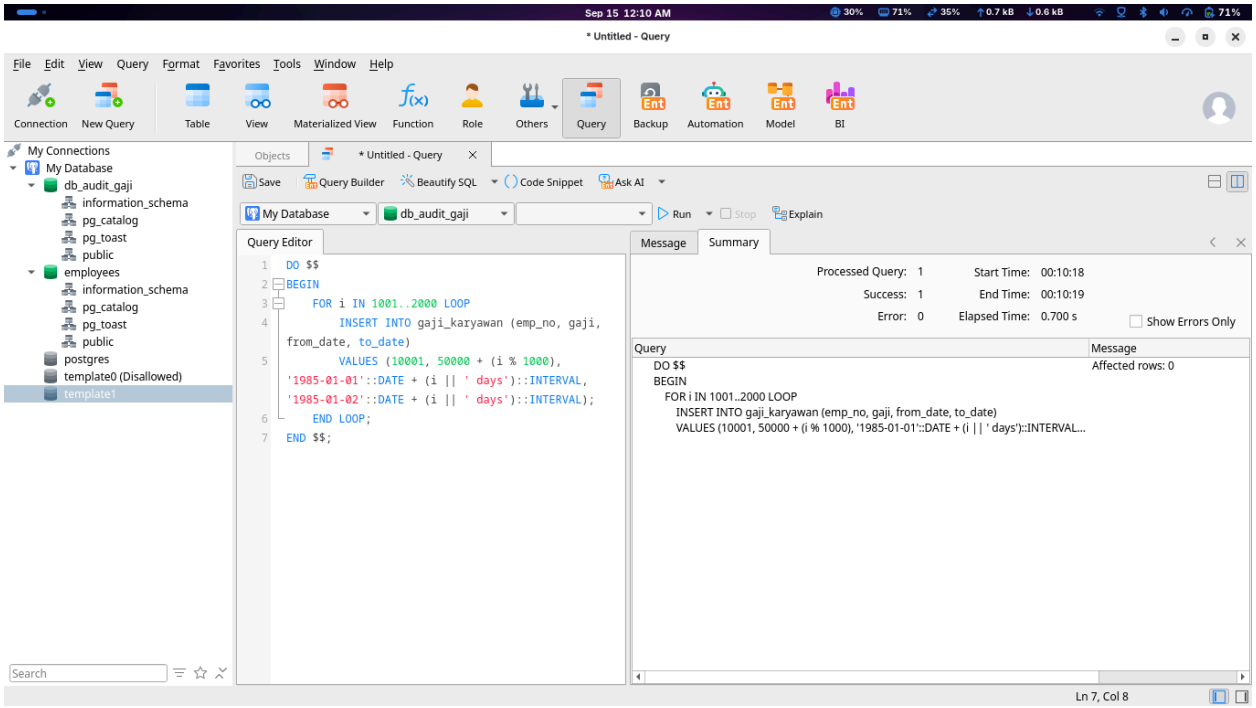
Tahap F.5 - Pengukuran Dampak Kinerja Trigger (Benchmark 1.000 Baris)

Membandingkan kecepatan input 1.000 data saat trigger audit nyala versus saat dimatikan.

Pengujian Kondisi A - Seluruh Trigger Aktif



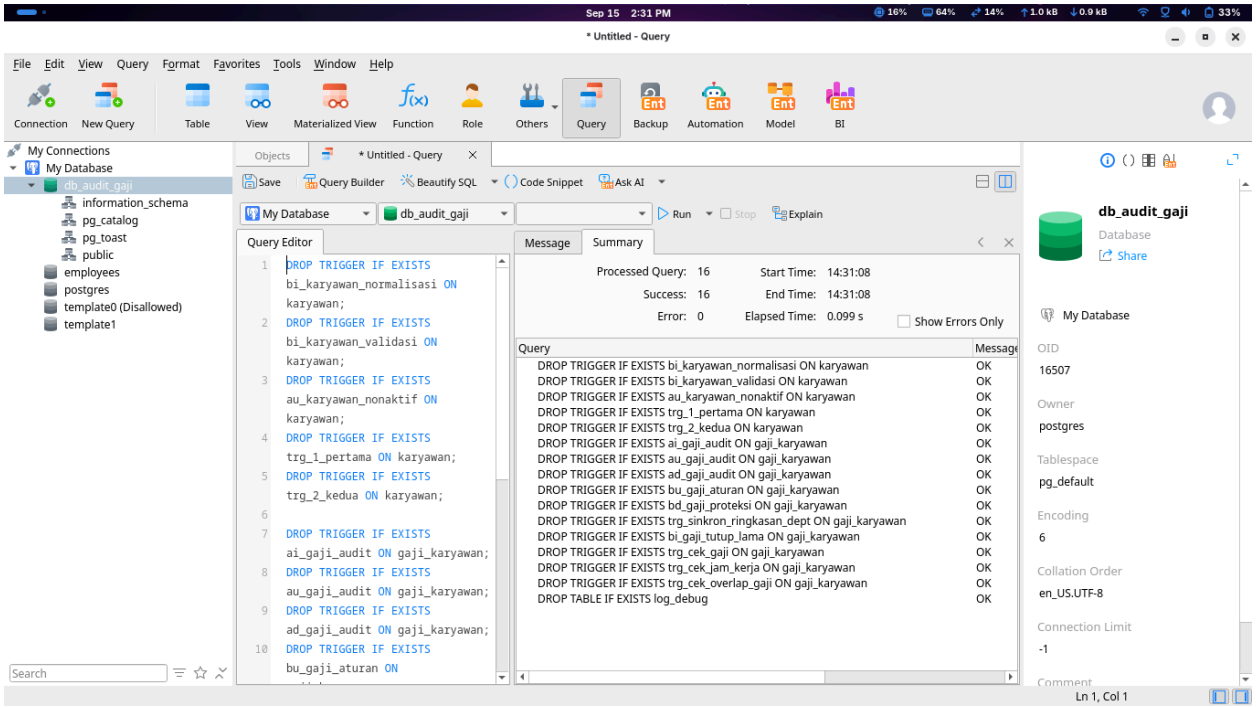
Pengujian Kondisi B - Trigger Audit Dinonaktifkan Sementara



Skenario Pengujian (1.000 Baris)	Waktu Eksekusi (Elapsed Time)	Status Trigger Audit
Kondisi A	0.515 s	Aktif (ai_gaji_audit)
Kondisi B	0.700 s	Dinonaktifkan (DISABLE TRIGGER)

Seharusnya proses input data saat trigger mati berjalan lebih cepat karena database tidak perlu menulis catatan tambahan ke tabel audit. Perbedaan waktu pada pengujian 1.000 data ini masih tipis karena dipengaruhi beban memori dan penulisan log transaksi database saat pengujian berjalan.

Tahap F.6 - Skrip Pembersihan (*Cleanup Script*)
Menghapus semua trigger dan tabel tambahan agar database kembali bersih seperti awal.



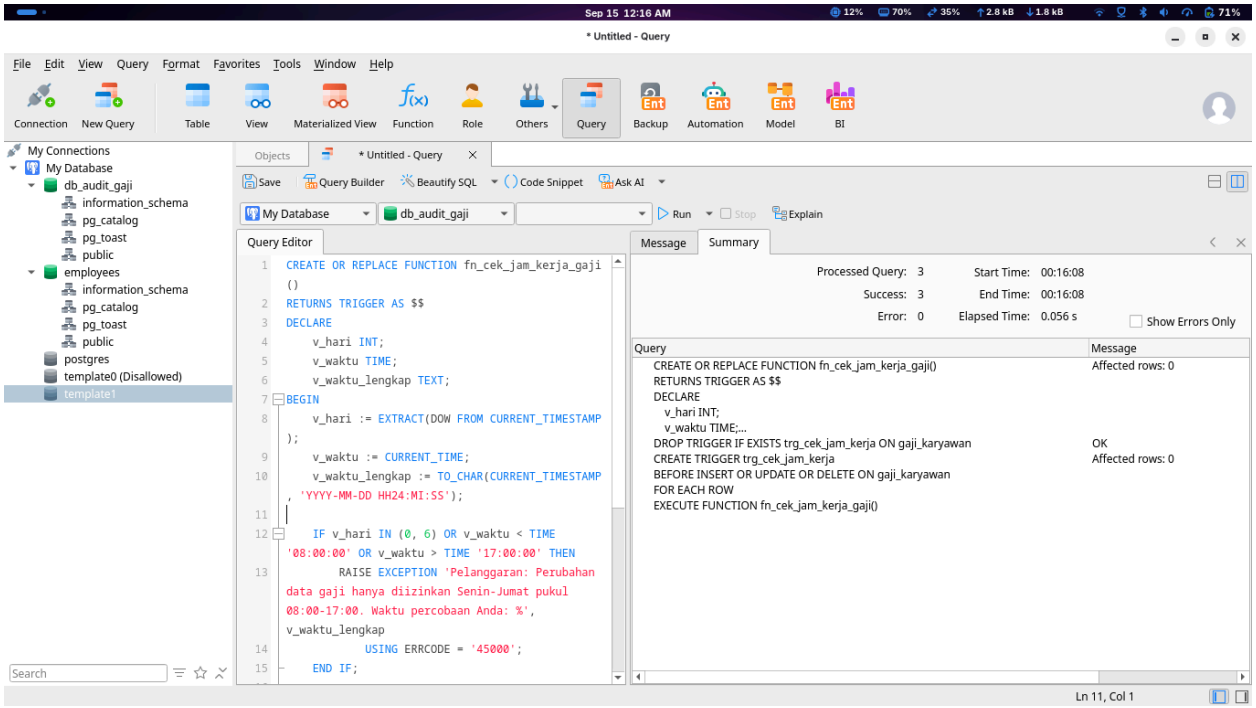
Mengapa debugging trigger lebih rumit dibanding kode aplikasi biasa? Karena trigger berjalan langsung di background database tanpa tampilan konsol untuk mencetak log (seperti print atau console.log). Selain itu, saat terjadi error, database langsung melakukan rollback sehingga data debug yang sempat dimasukkan ikut terhapus.

Dua kebiasaan kerja untuk mempermudah debugging trigger:

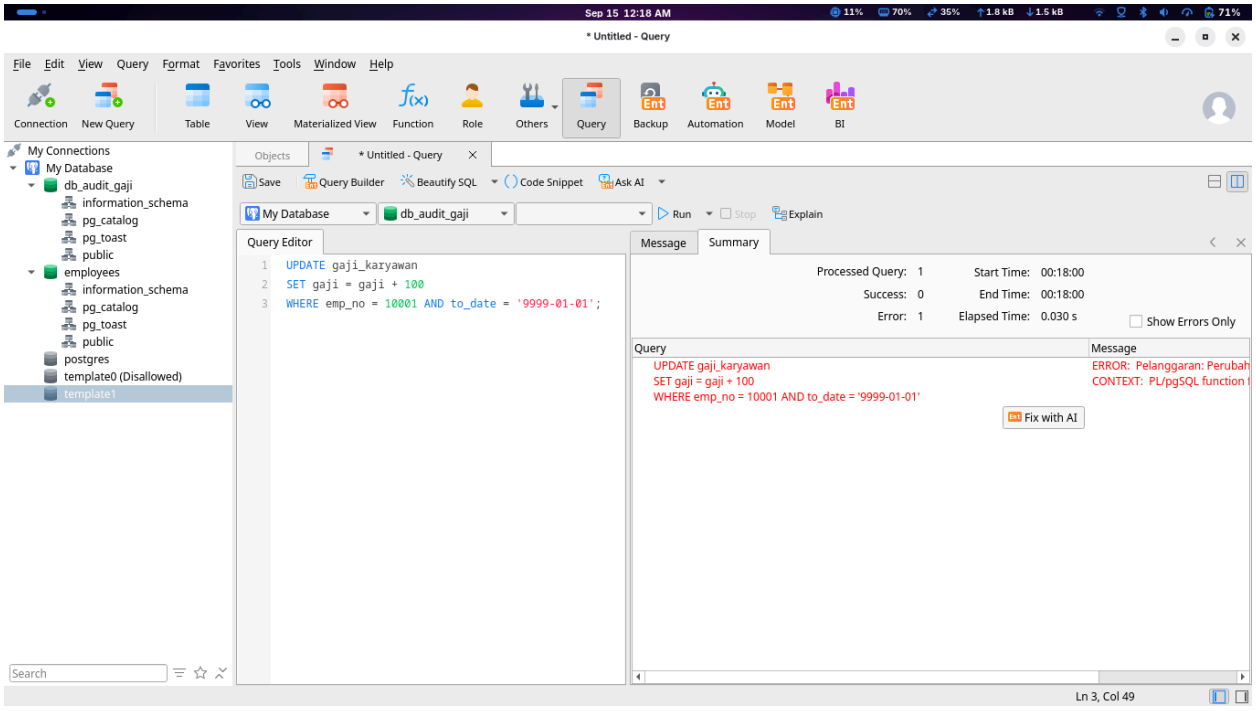
1. Membuat tabel penampung khusus (seperti tabel log_debug) untuk memantau nilai variabel sebelum error dilempar.
2. Memisahkan logika pengecekan yang rumit ke fungsi terpisah agar bisa diuji secara mandiri sebelum digabungkan ke trigger.

Bagian G - Tantangan / Pengayaan
Tahap G.1 - Pembatasan Hari dan Jam Kerja
Mengunci perubahan gaji agar cuma bisa dilakukan pada hari kerja dan jam kerja kantor.

1. Buat Fungsi dan Trigger Pembatasan Jam Kerja



Pengujian Tahap G.1 - Menguji Pembatasan Waktu

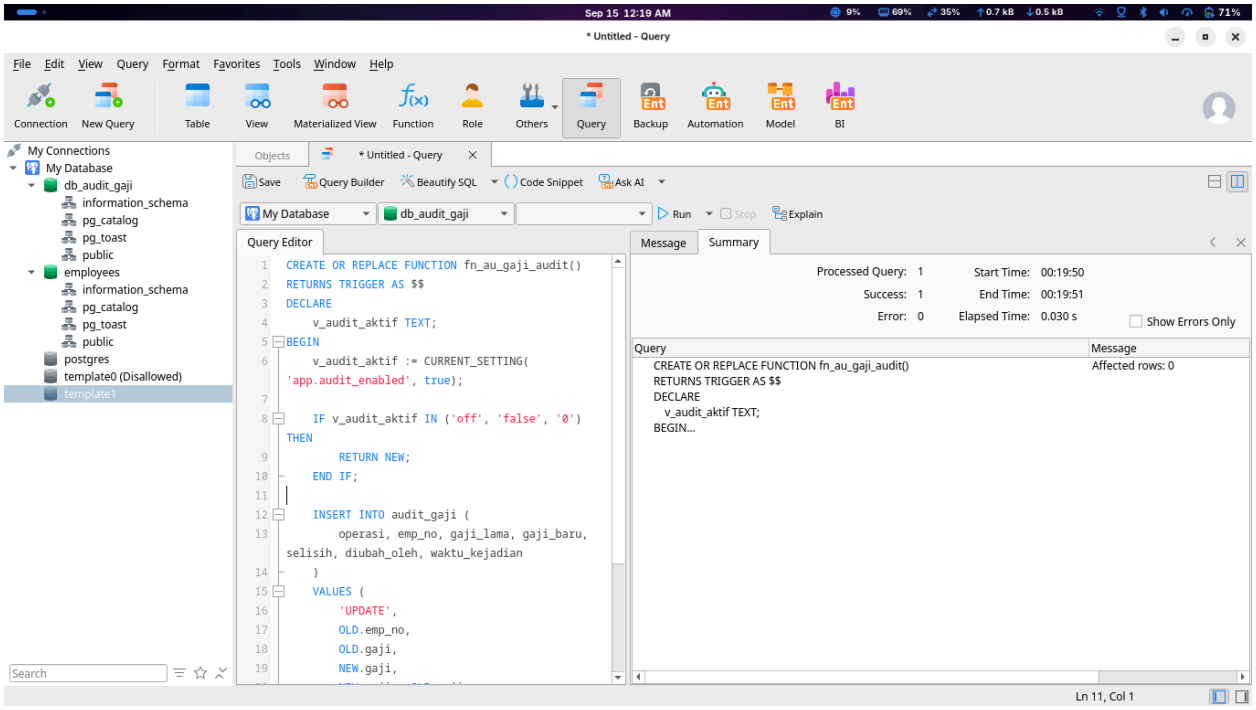


Tahap G.2 - Mekanisme Saklar Audit (*Audit Toggle*)

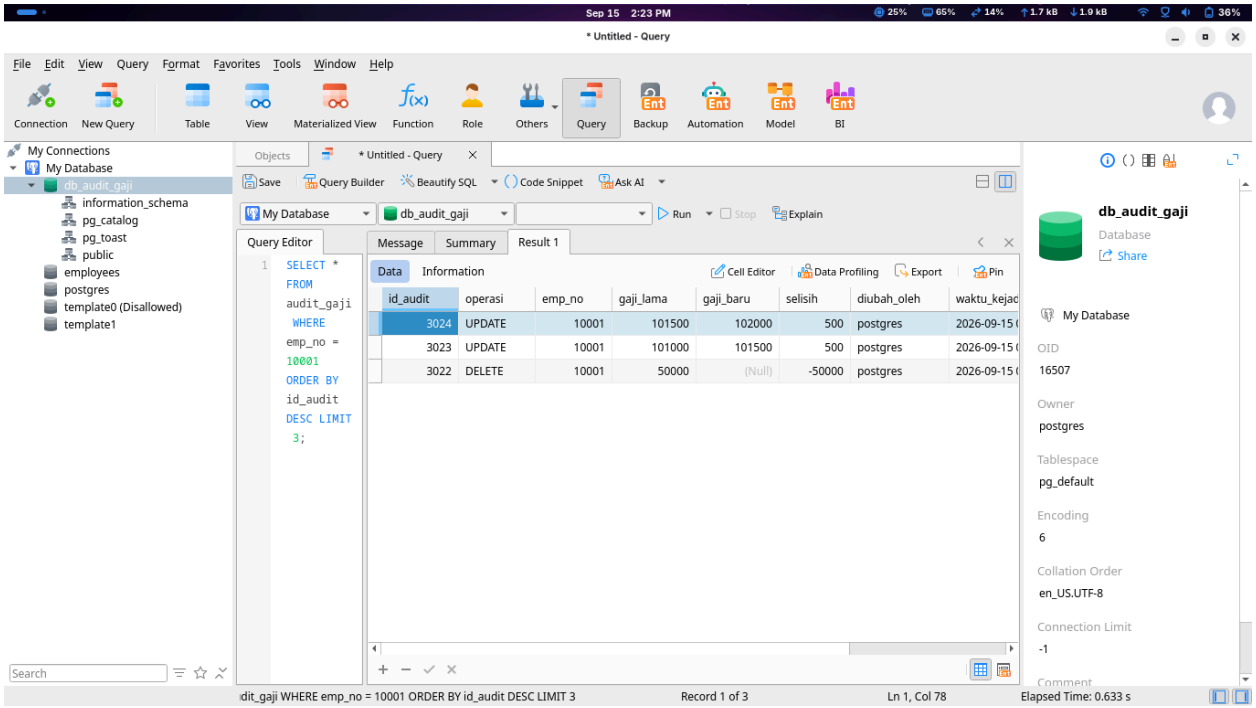
Menyiapkan saklar on/off agar sistem audit bisa dimatikan sementara saat ada update data dalam

jumlah banyak.

1. Perbarui Fungsi Audit Gaji dengan Pengecekan Saklar Sesi

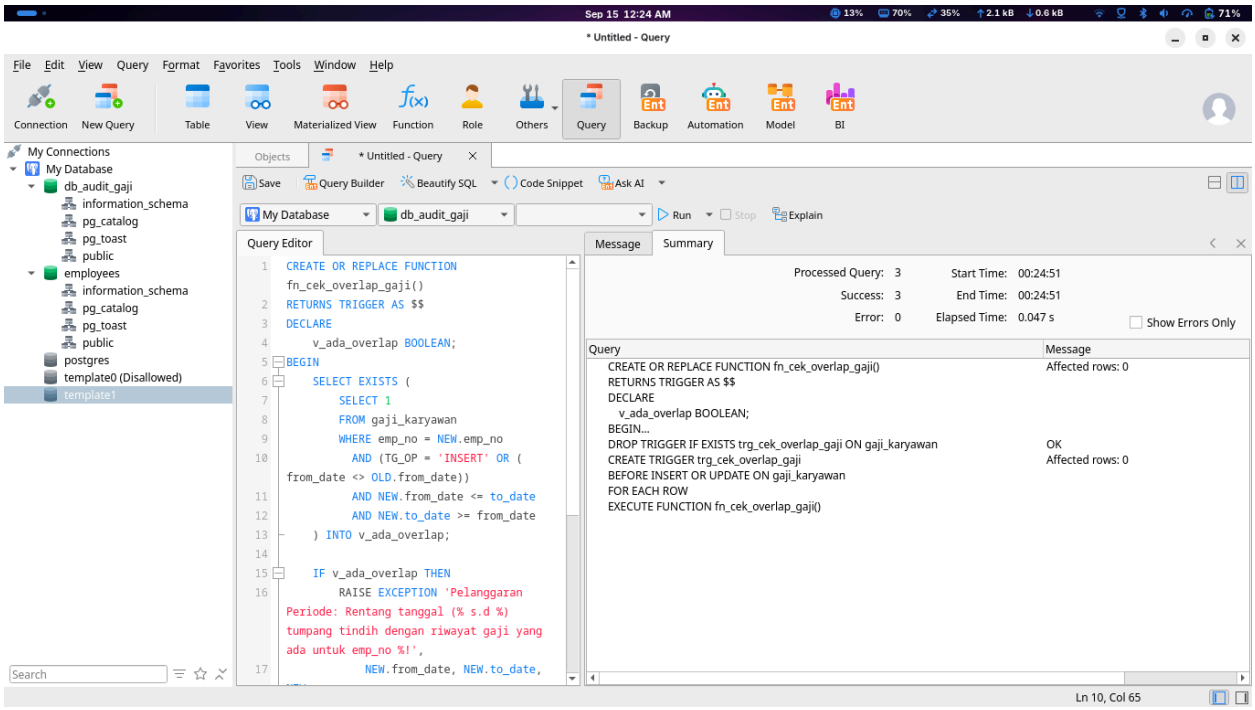


2. Uji Coba Mode Saklar Nonaktif (*Batch Maintenance Mode*)

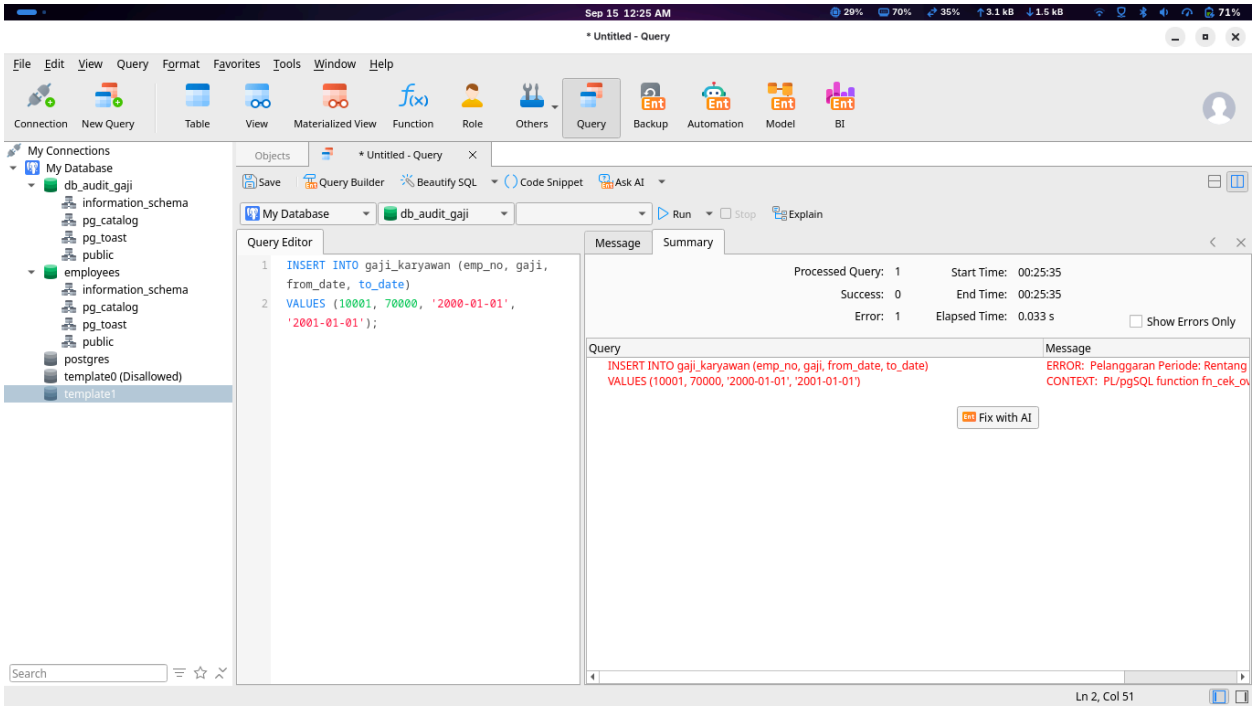


Tahap G.3 - Pencegahan Tumpang Tindih Periode Gaji (*Temporal Overlap Protection*)
Menjaga agar masa berlaku gaji karyawan tidak saling tabrakan tanggalnya.

1. Buat Fungsi dan Trigger Deteksi Overlap



2. Uji Coba Penolakan Overlap


Sep 15 12:25 AM
* Untitled - Query
File Edit View Query Format Favorites Tools Window Help
Connection
New Query
Table
View
Materialized View
Function
Role
Others
Query
Backup
Automation
Model
BI
My Connections
My Database
db_audit_gaji
information_schema
pg_catalog
pg_toast
public
employees
information_schema
pg_catalog
pg_toast
public
postgres
template0 (Disallowed)
template1
Search
Objects
* Untitled - Query
Save
Query Builder
Beautify SQL
Code Snippet
Ask AI
My Database
db_audit_gaji
Run
Stop
Explain
Query Editor
1 INSERT INTO gaji_karyawan (emp_no, gaji,
from_date, to_date)
2 VALUES (10001, 70000, '2000-01-01',
'2001-01-01');
Message
Summary
Processed Query: 1
Start Time: 00:25:35
Success: 0
End Time: 00:25:35
Error: 1
Elapsed Time: 0.033 s
Show Errors Only
Query
Message
INSERT INTO gaji_karyawan (emp_no, gaji, from_date, to_date)
VALUES (10001, 70000, '2000-01-01', '2001-01-01')
ERROR: Pelanggaran Periode: Rentang
CONTEXT: PL/pgSQL function fn_cek_ov
Fix with AI
Ln 2, Col 51